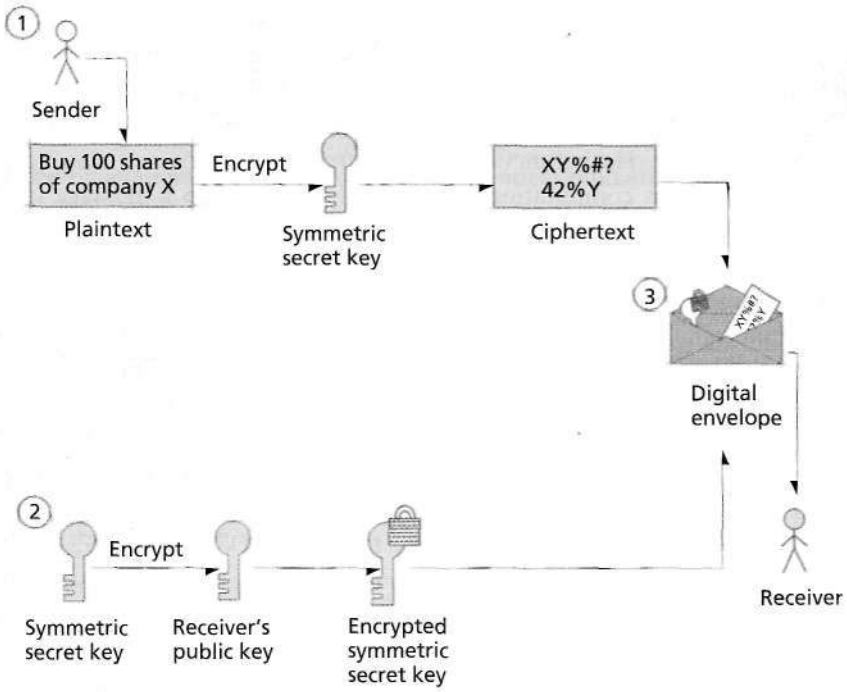

Figure 19.9 | *Creating a digital envelope.*

An important aspect of key management is **key generation**—the process by which keys are created. A malicious third party could try to decrypt a message by using every possible decryption key. Key-generation algorithms are sometimes unintentionally constructed to choose from only a small subset of possible keys. If the subset is too small, then the encrypted data is more susceptible to brute-force attacks. Therefore, it is important to use a key-generation program that can generate a large number of keys as randomly as possible. Key security is improved when key length is large enough that brute-force cracking is computationally infeasible.

Self Review

1. Why is it important to use a long key length?
2. Why are most security breaches due to poor key management rather than cryptanalytic attack?

Ans: **1)** If the total number of decryption keys is small, malicious third parties could generate all possible decryption keys quickly to break the encryption. **2)** When long key lengths are used, it is easier to take advantage of poor key management to steal private keys than it is to use a brute-force method such as a cryptanalytic attack.

19.8.2 Digital Signatures

Digital signatures, the electronic equivalents of written signatures, were developed to address the absence of authentication and integrity in public-key cryptography. A digital signature authenticates the sender's identity and is difficult to forge.

To create a digital signature, a sender first applies a hash function to the original plaintext message. Hash functions for secure applications are also typically designed such that it is computationally infeasible to compute a message from its hash value or to generate two messages with the same hash value. A secure hash algorithm is designed so that the probability that two different messages will produce the same message digest is statistically insignificant.

In digital signatures, the hash value uniquely identifies a message. If a malicious party changes the message, the hash value also changes, thus enabling the recipient to detect that the message has been altered. Two widely used hashing functions are the **Secure Hash Algorithm (SHA-1),** developed by National Institute of Standards and Technology (NIST), and the **MD5 Message Digest Algorithm.** developed by Professor Ronald L. Rivest at MIT. Using SHA-f, the phrase *"Buy 100 shares of company X"* would produce the hash value *D8A9 B6 9F 72 65 OB D5 6D OC 47 00 95 OD FD 31 96 OA FD B5.* The hash value is called the **message digest.**

Next, the sender uses the sender's private key to encrypt the message digest. This step creates a digital signature and validates the sender's identity, because only the owner of that private key could encrypt the message. A message that includes the digital signature, hash function and the encrypted message is sent to the receiver. The receiver uses the sender's public key to decipher the digital signature (this establishes the message's authenticity) and reveal the message digest. The receiver then uses his or her own private key to decipher the original message. Finally, the receiver applies the hash function to the original message. If the hash value of the original message matches the message digest included in the signature, the **message integrity** is confirmed—the message has not been altered in transmission.

Digital signatures are significantly more secure than handwritten signatures. A digital signature is based upon the contents of the document, so a digital signature (contrary to a handwritten signature) is different for each document signed.

Digital signatures do not provide proof that a message has been sent. Consider the following situation: A contractor sends a company a digitally signed contract, which the contractor later would like to revoke. The contractor could do so by releasing the private key, then claiming that the digitally signed contract was generated by an intruder who stole the contractor's private key. **Time stamping,** which binds a time and date to a digital document, can help solve the problem of nonrepudiation. For example, suppose the company and the contractor are negotiating a contract. The company requires the contractor to sign the contract digitally and then have the document digitally time stamped by a third party, called a **time-stamping agency** or a **digital notary service.**

The contractor sends the digitally signed contract to the time-stamping agency. The privacy of the message is maintained, because the time-stamping agency sees

only the encrypted, digitally signed message. The agency affixes the time and date of receipt to the encrypted, signed message and digitally signs the entire package with the time-stamping agency's private key. The time-stamp cannot be altered by anyone except the time-stamping agency, because no one else possesses the agency's private key. Unless the contractor reports that the private key was compromised before the document was time stamped, the contractor cannot legally prove that the document was signed by an unauthorized third party. The sender could also require the receiver to sign the message digitally and time stamp it as proof of receipt. To learn more about time-stamping, visit www.authentidate.com.

The U.S. government's digital authentication standard is called the **Digital Signature Algorithm (DSA).** Legislation passed in June of 2000 allows digital signatures to be as legally binding as handwritten signatures, which has facilitated many e-business transactions. For the latest news about U.S. government legislation in information security, visit www.itaa.org/infosec.

Self Review

1. If digital signatures do not create a message digest, can a malicious third party modify a message's ciphertext?
2. When interpreting messages that are digitally signed, what is the difference between authenticating a message and verifying its integrity?

Ans: **1)** Although it is possible to modify the ciphertext, it is unlikely that the third party will be able to modify the message in any meaningful way without knowledge of the encryption key. **2)** To authenticate a message, the receiver uses the sender's public key to decipher the digital signature. To verify a message's integrity, the receiver checks the hash value of the original message with the message digest included in the signature.

19.9 Public-Key Infrastructure, Certificates and Certificate Authorities

A limitation of public-key cryptography is that multiple users might share the same set of keys, making it difficult to establish each party's identity. For example, suppose a customer wishes to place an order with an online merchant. How does the customer know that the Web site being viewed was created by that merchant and not by a third party masquerading as a merchant to steal credit card information? **Public Key Infrastructure (PKI)** provides a solution to such a problem by integrating public-key cryptography with digital certificates and certificate authorities to authenticate parties in a transaction.

A **digital certificate** is a digital document that identifies a user and is issued by a **certificate authority (CA).** A digital certificate includes the name of the subject (the company or individual being certified), the subject's public key, a serial number (which uniquely identifies the certificate), an expiration date, the signature of the trusted certificate authority and any other relevant information. A CA can be a financial institution or any other trusted third party, such as VeriSign. Once issued, the CA makes the digital certificates publicly available in **certificate repositories.**

The CA signs the certificate by encrypting either the subject's public key or a hash value of the public key using the CA's own private key, so that a recipient can verify the certificate. The CA must verify every subject's public key. Thus, users must trust the public key of a CA. Usually, each CA is part of a **certificate authority hierarchy.** This hierarchy can be viewed as a tree structure in which each node relies on its parent for authentication information. The root of the certificate authority hierarchy is the Internet Policy Registration Authority (IPRA). The IPRA signs certificates using the **root key,** which signs certificates exclusively for **policy creation authorities**—organizations that set policies for obtaining digital certificates. In turn, policy creation authorities sign digital certificates for CAs. CAs then sign digital certificates for individuals and organizations.

The CA is responsible for authentication, so it must check user information carefully before issuing a digital certificate. In one case, human error caused Verisign to issue two digital certificates to an imposter posing as a Microsoft employee.[110] Such an error can be significant; for example, the inappropriately issued certificates can cause users to download malicious code unknowingly onto their machines. VeriSign, Inc., is a leading certificate authority (www.verisi gn. com); other digital certificate vendors are listed in the Web Resources section at the end of the chapter.

Digital certificates are created with an expiration date to force users to refresh key pairs periodically. If a private key is compromised before its expiration date, the digital certificate can be cancelled, and the user can obtain a new key pair and digital certificate. Cancelled and revoked certificates are placed on a **certificate revocation list (CRL),** which is maintained by the certificate authority that issued the certificates.

CRLs are analogous to the paper lists of revoked credit card numbers formerly used at the points of sale in retail stores.[111] This can substantially lengthen the process of determining the validity of a certificate. An alternative to CRLs is the **Online Certificate Status Protocol (OCSP),** which validates certificates in real time.[112]

Many still consider e-commerce unsecure. However, transactions using **PKI** and digital certificates can be more secure than exchanging private information over public, unencrypted media such as voice phone lines, paying through the mail, or handing a credit card to a sales clerk. In contrast, the key algorithms used in most secure online transactions are nearly impossible to compromise. By some estimates, the key algorithms used in public-key cryptography are so secure that even millions of today's computers working in parallel could not break the codes after a century of dedicated computation. However, as computing power increases, key algorithms considered strong today could be easily compromised in the future.

Self Review

1. Describe how CAs ensure the security of their certificates.
2. Why are electronic transactions secured by PKI and digital certificates often more secure than those conducted over the phone or in person?

Ans: **1)** Each CA is part of a certificate authority hierarchy. This hierarchy can be viewed as a tree structure in which each node relies on its parent for authentication information. Its root is the Internet Policy Registration Authority (IPRA), which signs certificates for policy creation authorities. Policy creation authorities sign digital certificates for CAs. CAs then sign digital certificates for individuals and organizations. **2)** The reason is that it is computationally infeasible to compromise the key algorithms used in most secure online transactions. By some estimates, the key algorithms used in public-key cryptography are so secure that even millions of today's computers working in parallel could not break the codes after a century of dedicated computation. By comparison, it is far easier to tap a phone line or write down an individual's credit card number during an in-person transaction.

19.10 Secure Communication Protocols

Secure communication protocols have been developed for several layers of the traditional TCP/IP stack. In this section we discuss Secure Sockets Layer (SSL) and Internet Protocol Security (IPSec). We also discuss the protocols that are commonly used to implement secure wireless communication.

19.10.1 Secure Sockets Layer (SSL)

The **Secure Sockets Layer (SSL)** protocol, developed by Netscape Communications, is a nonproprietary protocol that secures communication between two computers on the Internet and the Web.[113] Many e-businesses use SSL for secure online transactions. Support for SSL is included in many Web browsers, such as Mozilla (a stand-alone browser that has been integrated into Netscape Navigator), Microsoft's Internet Explorer and other software products. It operates between the Internet's TCP/IP communications protocol and the application software.[114]

During typical Internet communication, a process sends and receives messages via a socket. Although TCP guarantees that messages are delivered, it does not determine whether packets have been maliciously altered during transmission. For example, an attacker can change a packet's source and destination addresses without being detected, so fraudulent packets can be disguised as valid ones.[115]

SSL implements public-key cryptography using the RSA algorithm and digital certificates to authenticate the server in a transaction and to protect private information as it passes over the Internet. SSL transactions do not require client authentication; many servers consider a valid credit card number to be sufficient for authentication in secure purchases. To begin, a client sends a message to a server to open a socket. The server responds and sends its digital certificate to the client for authentication. Using public-key cryptography to communicate securely, the client and server negotiate session keys to continue the transaction. Session keys are secret keys that are used for the duration of that transaction. Once the keys are established, the communication is secured and authenticated using the session keys and digital certificates.

Before sending a message over TCP/IP, the SSL protocol breaks the information into blocks, each of which is compressed and encrypted. When the receiver obtains the data through TCP/IP, SSL decrypts the packets, then decompresses and

assembles the data. SSL is used primarily to secure point-to-point connections—interactions between exactly two computers.[116] SSL can authenticate the server, the client, both or neither. However, in most e-business SSL sessions, only the server is authenticated.

The Transport Layer Security (TLS) protocol, designed by the Internet Engineering Task Force (IETF), is the successor to SSL. Conceptually, SSL and TLS are nearly identical—they authenticate using symmetric keys and encrypt data with private keys. The difference between the two protocols is in the implementation of the algorithm and structure of TLS packets.[117]

Although SSL protects information as it is passed over the Internet, it does not protect private information, such as credit card numbers, once the information is stored on the merchant's server. Credit card information may be decrypted and stored in plaintext on the merchant's server until the order is placed. If the server's security is breached, an unauthorized party can access the information.

Self Review

1. Why do many transactions via SSL authenticate only the server?
2. Why is information compressed before encryption under SSL?

Ans: **1)** The reason is that many servers consider a valid credit card number to be sufficient for authentication in secure purchases. **2)** One reason is that compressed data consumes less space, thus requiring less time to transmit over the Internet. Another reason is that compression makes it more difficult for an attacker to interpret the unencrypted message, which provides greater protection against a cryptanalytic attack.

19.10.2 Virtual Private Network (VPMs) and IP Security (IPSec)

Most home or office networks are private—computers in the network are connected via communication lines not intended for public use. Organizations can also use the existing infrastructure of the Internet to create **Virtual Private Networks (VPNs),** which provide secure communication channels over public connections. Because VPNs use the existing Internet infrastructure, they are more economical than wide-area private networks.[118] Encryption enables VPNs to provide the same services and security as private networks.

A VPN is created by establishing a secure communication channel, called a tunnel, through which data passes over the Internet. **Internet Protocol Security (IPSec)** is commonly used to implement a secure tunnel by providing data privacy, integrity and authentication.[119] IPSec, developed by the IETF, uses public-key and symmetric-key cryptography to ensure data integrity, authentication and confidentiality.

The technology builds upon the Internet Protocol (IP) standard, whose primary security weakness is that IP packets can be intercepted, modified and retransmitted without detection. Unauthorized users can access the network by using a number of well-known techniques, such as **IP spoofing**—whereby an attacker simulates the IP address of an authorized user or host to obtain unauthorized access to resources. Unlike the SSL protocol, which enables secure, point-to-point connec-

tions between two applications, IPSec secures connections among all users of the network on which it is implemented.

The Diffie-Hellman and RSA algorithms are commonly used for key exchange in the IPSec protocol, and DES or 3DES are used for secret-key encryption, depending on the system and its encryption needs. *[Note:* The Diffie-Hellman encryption algorithm is a public-key algorithm.][120] IPSec encrypts packet data, called a payload, then places it in an unencrypted IP packet to establish the tunnel. The receiver discards the IP packet header, then decrypts the payload to access the packet's content.[121]

VPN security relies on three concepts—user authentication, data encryption and controlled access to the tunnel.[122] To address these three security issues, IPSec is composed of three components. The **authentication header (AH)** attaches information (such as a checksum) to each packet to verify the identity of the sender and prove that data was not modified in transit. The **encapsulating security payload (ESP)** encrypts the data using symmetric keys to protect it from eavesdroppers while the IP packet is transmitted across public communication lines. IPSec uses the **Internet Key Exchange (IKE)** protocol to perform key management, which allows secure key exchange.

VPNs are becoming increasingly popular in businesses, but remain difficult to manage. The network layer (i.e., IP) is typically managed by the operating system or hardware, not user applications. So, it can be difficult to establish a VPN because network users must install the proper software or hardware to support IPSec. For more information about IPSec, visit the IPSec Developers Forum at www.ipsec.com and the IETF's IPSec Working Group at www.ietf.org/html.charters/ipsec-charter.html.

Self Review

1. Why do companies often choose VPNs over private WANs?
2. Why must the encrypted IP packet be sent inside an unencrypted IP packet?

Ans: **1)** Because VPNs use the existing infrastructure of the Internet, VPNs are much more economical than WANs. **2)** Existing routers must be able to forward the packet. If the entire IP packet were encrypted, routers could not determine its destination.

19.10.3 Wireless Security

As wireless devices become increasingly popular, transactions such as stock trading and online banking are commonly transmitted wirelessly. Unlike wired transmissions, which require a third party to physically tap into a network, wireless transmissions enable virtually anyone to access transmitted data. Because wireless devices exhibit limited bandwidth and processing power, high latency and unstable connections, establishing secure wireless communication can be challenging.[123]

The IEEE 802.11 standard, which defines specifications for wireless LANs, uses the **Wired Equivalent Privacy (WEP)** protocol to protect wireless communication. WEP provides security by encrypting transmitted data and preventing unau-

thorized access to the wireless network. To transmit a message, WEP first appends a checksum to the message to enable the receiver to verify message integrity. It then applies the RC4 encryption algorithm, which takes two arguments—a 40-bit secret key (shared between the sender and the receiver) and a random 24-bit value (called the initialization value)—and returns a keystream (a string of bytes). Finally, WEP encodes the message by XORing the keystream and the plain text. Each WEP packet contains the encrypted message and the unencrypted initialization value. Upon receiving a packet, the receiver extracts the initialization value. The receiver then uses the initialization value and the shared secret key to generate the keystream. Finally, it decodes the message by XORing the keystream and the encrypted message. To determine whether the contents of the packet have been modified, the receiver can calculate the checksum and compare it with the checksum appended to the message.[124]

The security provided by the WEP protocol is inadequate for many environments. For one, the 24-bit initialization value is small, causing WEP to use the same keystream to encode different messages, which can allow a third party to recover plaintext relatively easily. WEP also exhibits poor key management—the secret key can be cracked by brute-force attacks within hours using a personal computer. Most WEP-enabled wireless networks share a single secret key between every client on the network, and the secret key is often used for months and even years. Finally, the checksum computation uses simple arithmetic, enabling attackers to change both the message body and the checksum with relative ease.[125, 126]

To address these issues, the IEEE and the Wi-Fi Alliance (www.weca.net/OpenSection/index.asp) developed a specification called **Wi-Fi Protected Access (WPA)** in 2003. WPA is expected to replace WEP by providing improved data encryption and by enabling user authentication, a feature not supported by WEP WPA also introduces dynamic key encryption, which generates a unique encryption key for each client. Authentication is achieved via an authentication server that stores user credentials.[127]

Both WEP and WAP are designed for communication using the 802.11 wireless standard. Other wireless technologies —such as Cellular Digital Packet Data (CDPD) for cell phones, PDAs and pagers, Third Generation Global System for Mobile Communications (3GSM) for 3GSM-enabled cell phones, PDAs and pagers, and Time Division Multiple Access (TDMA) for TDMA cell phones—define their own (often proprietary) wireless communication and security protocols.[128]

Self Review

1. What properties of wireless devices introduce challenges when implementing secure wireless communication?
2. How does WPA address the poor key management in WEP?

Ans: **1)** Limited bandwidth, limited processing power, high latency and unstable connections. **2)** WPA introduces dynamic key encryption, which generates a unique encryption key for each client.

19.11 Steganography

Steganography is the practice of hiding information within other information, derived from Greek roots meaning "covered writing." Like cryptography, steganography has been used since ancient times. Steganography can be used to hide a piece of information, such as a message or image, within another image, message or other form of multimedia.

Consider a simple textual example: Suppose a stock broker's client wishes to perform a transaction via communication over an unsecure channel. The client might send the message "BURIED UNDER YARD." If the client and broker agreed in advance that the message is contained in the first letters of each word, the stock broker extracts the message, "BUY."

An increasingly popular application of steganography is **digital watermarks** for intellectual property protection. Digital steganography exploits unused portions of files encoded using particular formats, such as in images or on removable disks. The insignificant space stores the hidden message, while the digital file maintains its intended semantics.[129] A digital watermark can be either visible or invisible. It is usually a company logo, copyright notification or other mark or message that indicates the owner of the document. The owner of a document could show the hidden watermark in a court of law, for example, to prove that the watermarked item was stolen.

Digital watermarking could have a substantial impact on e-commerce. Consider the music industry. Music and motion picture distributors are concerned that MP3 (compressed audio) and MPEG-4 (compressed video) technologies facilitate the illegal distribution of copyrighted material. Consequently, many publishers are hesitant to publish content online, as digital content is easy to copy. Also, because CD- and DVD-ROMs store digital information, users are able to copy multimedia files and share them via the Web. Using digital watermarks, music publishers can make indistinguishable changes to a part of a song at a frequency that is not audible to humans, to show that the song was, in fact, copied.

Blue Spike's Giovanni™ digital watermarking software uses cryptographic keys to generate and embed steganographic digital watermarks into digital music and images (www.bluespike.com). The watermarks are placed randomly and are undetectable without knowledge of the embedding scheme; thus, they are difficult to identify and remove.

Self Review

1. Compare and contrast steganography and cryptography.
2. In grayscale bitmap images, each pixel can be represented by a number between 0 and 255. The number represents a certain shade of gray between 0 (black) and 255 (white). Describe a possible steganography scheme, and explain why it works.

Ans: **1)** Steganography and cryptography are similar because they both involve sending a message privately. However, with steganography, third parties are unaware that a secret message has been sent, though the message is typically in plaintext. With cryptography, third parties are aware that a secret message is sent, but they cannot decipher the message easily. **2)** A

secret message can be encoded by replacing each of the least significant bits of the picture with one bit of the message. This would not affect the picture much, because each pixel would become at most one shade lighter or darker. In fact, it would be undetectable by most third parties that the picture contained a hidden message.

19.12 Proprietary and Open-Source Security

When comparing the merits of open-source and proprietary software, security is an important concern. Supporters of open-source security claim that proprietary solutions rely on security through obscurity. Proprietary solutions assume that because attackers cannot analyze the source code for security flaws, fewer security attacks are possible.

Supporters of open-source security claim that proprietary security applications limit the number of collaborative users that can search for security flaws and contribute to the overall security of the application. If the public cannot review the software's source code, certain bugs that are overlooked by developers can be exploited. Developers of proprietary software argue that their software has been suitably tested by their security experts.

Open-source security supporters also warn that proprietary security developers are reluctant to publicly disclose security flaws, even when accompanied by appropriate patches, for fear that public disclosure would damage the company's reputation. This may discourage vendors from releasing security patches, thereby reducing their product's security.

A primary advantage to open-source applications is interoperability—open-source software tends to implement standards and protocols that many other developers can easily include in their products. Because users can modify the source, they can customize the level of security in their applications. Thus, an administrator can customize the operating system to include the most secure access control policy file system, communication protocol and shell. By contrast, users of proprietary software often are limited to the features provided by the vendor, or must pay additional fees to integrate other features.

Another advantage to open-source security is that the application source code is available for extensive testing and debugging by the community at large. For example, the Apache Web server is an open-source application that survived four and half years without a serious vulnerability—an almost unrivaled feat in the field of computer security.[130]-[131] Open-source solutions can provide code that is provably secure before deployment. Proprietary solutions rely on an impressive security track record or recommendations from a small panel of experts to vouch for their security.

The primary limitation of security in open-source applications is that open-source software is often distributed without standard default security settings, which can increase the likelihood that security will be compromised due to human error, such as inexperience or negligence. For example, several Linux distributions enable many network services by default, exposing the computer to external attacks. Such services require significant customization and modification to prevent

skilled attackers from compromising the system. Proprietary systems, on the other hand, typically are installed with an appropriate security configuration by default. Unlike most UNIX-based distributions, the open-source OpenBSD operating system claims to install fully secured against external attacks by default. As any user will soon realize, however, significant configuration is required to enable all but the most basic of communication services.

It is important to note that proprietary systems can be as secure as open-source systems, despite opposition from the open-source community. Though each method offers relative merits and pitfalls, both require attention to security and timely releases of patches to fix any security flaws that are discovered.

Self Review

1. Why might a software development company be reluctant to admit security flaws?
2. How do open-source cryptography software solutions address the fact that the technique they use to generate keys is made public?

Ans: **1)** Acknowledgement of security flaws can damage a company's reputation. **2)** Such software must use a large set of decryption keys to prevent malicious third party brute-force attacks that attempt to compromise security by generating all possible decryption keys.

19.13 Case Study: INUX Systems Security

UNIX systems are designed to encourage user interaction, which can make them more difficult to secure.[132, 133, 134, 135, 136, 137, 138] UNIX systems are intended to be open systems; their specifications and source code are widely available.

The UNIX password file is encrypted. When a user enters a password, it is encrypted and compared to the encrypted password file. Thus, passwords are unrecoverable even by the system administrator. UNIX systems use salting when encrypting passwords.[139] The salting is a two-character string randomly selected via a function of the time and the process ID. Twelve bits of the salting then modify the encryption algorithm. Thus, users who choose the same password (by coincidence or intentionally) will have different encrypted passwords (with high likelihood). Some installations modify the password program to prevent users from choosing weak passwords.

The password file must be readable by any user because it contains other crucial information (i.e., usernames, user IDs, and the like) that is required by many UNIX tools. For example, because directories employ user IDs to record file ownership, *Is* (the tool that lists directory contents and file ownership) needs to read the password file to determine usernames from user IDs. If crackers obtain the password file, they could potentially break the password encryption. To address this issue, UNIX protects the password file from crackers by storing information other than the encrypted passwords in the normal password file and storing the encrypted passwords in a **shadow password file** that can be accessed only by users with root privileges.[140]

With the UNIX *setuid* permission feature, a program may be executed by one user using the privileges of another user. This powerful feature has security flaws,

particularly when the resulting privilege is that of the "superuser" (who has access to all files in a UNIX system).[141]-[142] For example, if a regular user is able to execute a shell belonging to the superuser, and for which the *setuid* bit has been set, then the regular user essentially becomes the superuser.[143] Clearly, *setuid* should be employed carefully. Users, including those with superuser privileges, should periodically examine their directories to confirm the presence of *setuid* files and detect any that should not be *setuid.*

A relatively simple means of compromising security in UNIX systems (and other operating systems) is to install a program that prints out the login prompt, copies what the user then types, fakes an invalid login and lets the user try again. The user has unwittingly given away his or her password! One defense is that if you are confident you typed the password correctly the first time, you should log into a different terminal and choose a new password immediately.[144]

UNIX systems include the *crypt* command, which allows a user to enter a key and plaintext; ciphertext is output. The transformation can be reversed trivially with the same key. One problem with this is that users tend to use the same key repeatedly; once the key is discovered, all other files encrypted with this key can be read. Users sometimes forget to delete their plaintext files after producing encrypted versions. This makes discovering the key much easier.

Often, too many people are given superuser privileges. Restricting superuser privileges can reduce the risk of attackers gaining control of a system due to errors made by inexperienced users. UNIX systems provide a substitute user identity (*su*) command to enable users to execute shells with a different user's credentials. All *su* activity should be logged; this command lets any user who types a correct password of another user assume that user's identity, possibly even acquiring superuser privileges.

A popular Trojan horse technique is to install a fake *su* program, which obtains the user's password, e-mails it to the attacker and restores the regular *su* program.[145] Never allow others to have write permission for your files, especially for your directories; if you do, you're inviting someone to install a Trojan horse.

UNIX systems contain a feature called **password aging,** in which the administrator determines how long passwords are valid; when a password expires, the user receives a message and is asked to enter a new password.[146]-[147] There are several problems with this feature:

1. Users often supply easy-to-crack passwords.
2. The system often prevents a user from resetting to the old (or any other) password for a week, so the user can not strengthen a weak password.
3. Users often switch between only two passwords.

Passwords should be changed frequently. A user can keep track of all login dates and times to determine whether an unauthorized user has logged in (which means that his or her password has been learned). Logs of unsuccessful login attempts often store passwords, because users sometimes accidentally type their password when they mean to type their username.

Some systems disable accounts after a small number of unsuccessful login attempts. This is a defense against the intruder who tries all possible passwords. An intruder who has penetrated the system can use this feature to disable the account or accounts of users, including the system administrator, who might attempt to detect the intrusion.[148]

The attacker who temporarily gains superuser privileges can install a trap-door program with undocumented features. For example, someone with access to source code could rewrite the login program to accept a particular login name and grant this user superuser privileges without even typing a password.[149]

It is possible for individual users to "grab" the system, thus preventing other users from gaining access. A user could accomplish this by spawning thousands of processes, each of which opens hundreds of files, thus filling all the slots in the open-file table.[150] Installations can guard against this by setting reasonable limits on the number of processes a parent can spawn and the number of files that a process can open at once, but this in turn could hinder legitimate users who need the additional resources.

Self Review

1. Why should the *setuid* feature be used with caution?
2. Discuss how UNIX systems protect against unauthorized logins and how attackers can circumvent and exploit these mechanisms.

Ans: **1)** When a user executes the program, the user's individual ID or group ID or both are changed to that of the program's owner only while the program executes. If a regular user is able to execute a shell belonging to the superuser, and for which the *setuid* bit has been set, then the regular user essentially becomes the superuser. **2)** To protect against unauthorized logins, UNIX systems use password aging to force users to change passwords frequently, track user logins and disable accounts after a small number of unsuccessful login attempts. Attackers can obtain user passwords using Trojan horses, exploiting the *su* command and *setuid* feature and disabling an account (e.g., the system administrator's account) to prevent that user from responding to an intrusion.

Web Resources

www.nsa.gov/selinux/index.html
Lists documents and resources for Security-Enhanced Linux.

www.microsoft.com/windowsxp/security
Provides links for maintaining security in Windows XP systems.

www.sun.com/software/solaris/trustedsolaris/index.html
Provides information about the trusted Solaris operating environment.

www.computerworld.com/securitytopics/security/story/0,10801,69976,00.html
Discusses two security flaws in Windows NT and 2000 and how they were fixed.

people.freebsd.org/~jkb/howto.html
Discusses how to improve FreeBSD system security.

www.deter.com/unix
Lists links for UNIX-related security papers.

www.alw.nih.gov/Security/Docs/network-security.html
Provides an architectural overview of UNIX network security.

www.newsfactor.com/perl/story/7907.html
Compares Linux and Windows security.

itmanagement.earthweb.com/secu/article.php/641211
Discusses the future of operating systems security.

www-106.ibm.com/developerworks/security/library/l-sec/index.html?dwzone=security
Provides links to Linux security issues and how to address these issues.

www.securemac.com/
Provides security news for Apple systems.

www.linuxjournal.com/article.php?sid=1204
Describes how to create a Linux firewall using the TIS Firewall Toolkit.

www.rsasecurity.com
Homepage of RSA security.

www.informit.cotn/isapi/guide~security/seq_id~13/guide/content.asp
Discusses operating system security overview, operating system security weakness.

www.ftponline.com/wss/
Homepage of the Windows Server System magazine, which contains various security articles.

www.networkcomputing.com/1202/1202fldl.html
Provides a tutorial on wireless security.

www.cnn.com/SPECIALS/2003/wireless/interactive/wi reless101/index.html
Provides a list of wireless security technologies for computers, PDAs, cell phones and other wireless devices.

www.securitysearch.com
This is a comprehensive resource for computer security, with thousands of links to products, security companies, tools and more. The site also offers a free weekly newsletter with information about vulnerabilities.

www.w3.org/Security/Overview.html
The *W3C Security Resources* site has FAQs, information about W3C security and e-commerce initiatives and links to other security-related Web sites.

www.rsasecurity.com/rsalabs/faq
This site is an excellent set of FAQs about cryptography from RSA Laboratories, one of the leading makers of public-key cryptosystems.

www.authentidate.com
Authentidate provides timestamping to verify the security of a file.

www.itaa.org/infosec
The Information Technology Association of America (ITAA) InfoSec site has information about the latest U.S. government legislation related to information security.

www.securitystats.com
This computer security site provides statistics on viruses, Web defacements and security spending.

www.ieee-security.org/cipher.html
Cipher is an electronic newsletter on security and privacy from the Institute of Electrical and Electronics Engineers (IEEE) You can view current and past issues online.

www.scmagazine.com
SC Magazine has news, product reviews and a conference schedule for security events.

www.cdt.org/crypto
Visit the Center for Democracy and Technology for U.S. cryptography legislation and policy news.

csrc.nist.gov/encryption/aes
The official site for the AES includes press releases and a discussion forum.

www.pkiforum.org/
The PKI Forum promotes the use of the Public Key Infrastructure.

www.Verisign.com
VeriSign creates digital IDs for individuals, small businesses and large corporations. Check out its Web site for product information, news and downloads.

www.thawte.com
Thawte Digital Certificate Services offers SSL, developer and personal certificates.

www.belsign.be/
Belsign is the European authority for digital certificates.

www.interhack.net/pubs/fwfaq
This site provides an extensive list of FAQs on firewalls.

www.indentix.com
Identix specializes in fingerprinting systems for law enforcement, access control and network security. Using its fingerprint scanners, you can log on to your system, encrypt and decrypt files and lock applications.

www.keytronic.com
Key Tronic manufactures keyboards with fingerprint recognition systems.

www.ietf.org/html.charters/ipsec-charter.html
The IPSec Working Group of the Internet Engineering Task Force (IETF) is a resource for technical information related to the IPSec protocol.

www.vpnc.org
The Virtual Private Network Consortium has VPN standards, white papers, definitions and archives. VPNC also offers compatibility testing with current VPN standards.

www.outguess.org
Outguess is a freely available steganographic tool.

www.weca.net/OpenSection/index.asp
Wi-Fi alliance provides information about Wi-Fi and resources for setting up and securing a Wi-Fi network.

Summary

Computer security addresses the issue of preventing unauthorized access to resources and information maintained by computers. Computer security commonly encompasses guaranteeing the privacy and integrity of sensitive data, restricting the use of computer resources and providing resilience against malicious attempts to incapacitate the system. Protection entails mechanisms that shield resources such as hardware and operating system services from attack.

Cryptography focuses on encoding and decoding data so that it can be interpreted only by the intended recipients. Data is transformed by means of a cipher or cryptosystem. Modern cryptosystems rely on algorithms that operate on the individual bits or blocks (a group of bits) of data, rather than letters of the alphabet. Encryption and decryption keys are binary strings of a given length.

Symmetric cryptography, also known as secret-key cryptography, uses the same secret key to encrypt and decrypt a message. The sender encrypts a message using the secret key, then sends the encrypted message to the intended recipient, who decrypts the message using the same secret key. A limitation of secret-key cryptography is that before two parties can communicate securely, they must find a secure way to exchange the secret key.

Public-key cryptography solves the problem of securely exchanging symmetric keys. Public-key cryptography is asymmetric in that it employs two inversely related keys: a public key and a private key. The private key is kept secret by its owner, while the public key is freely distributed. If the public key encrypts a message, only the corresponding private key can decrypt it.

The most common authentication scheme is simple password protection. The user chooses a password, memorizes it and presents it to the system to gain admission to a resource or system. Password protection introduces several weaknesses to a secure system. Users tend to choose passwords that are easy to remember, such as the name of a spouse or pet. Someone who has obtained personal information about the user might try to log in several times using passwords that are characteristic of the user; several repeated attempts might result in a security breach.

Biometrics uses unique personal information—such as fingerprints, eyeball iris scans or face scans—to identify a user. A smart card is often designed to resemble a credit card and can serve many different functions, from authentication to data storage. The most popular smart cards are memory cards and microprocessor cards.

Single sign-on systems simplify the authentication process by allowing the user to log in once using a single password. Users authenticated via a single sign-on system can then access multiple applications across multiple computers. It is important to secure single sign-on passwords, because if a password becomes available to hackers, all applications protected by that password can be accessed and attacked. Single sign-on systems are available in forms of workstation login scripts, authentication server scripts and token-based authentication.

The key to operating system security is to control access to system resources. The most common access rights are read, write and execute. Techniques that an operating system employ to manage access rights include the access control matrices, access control lists and capability lists.

There are several types of security attacks against computer systems, including cryptanalytic attacks, viruses and worms (such as the ILOVEYOU virus and Sapphire/Slammer worm), denial-of-service attacks (such as a domain name system (DNS) attack), software exploitation (such as buffer overflow) and system penetration (such as Web defacing).

Several common security solutions include firewalls (including packet-filtering firewall and application-level gateways), intrusion detection systems (including host-based intrusion detection and network-based intrusion detection), antivirus software using signature scanning and heuristic scanning, security patches (such as hotfixes) and secure file systems (such as the Encrypting File System).

The five fundamental requirements for a successful, secure transaction are privacy, integrity, authentication, authorization and nonrepudiation. The privacy issue deals with ensuring that the information transmitted over the Internet has not been viewed by a third party. The integrity issue deals with ensuring that the information sent or received has not been altered. The authentication issue deals with verifying the identities of the sender and receiver. The authorization issue deals with managing access to protected resources on the basis of user credentials. The nonrepudiation issue deals with ensuring that the network will operate continuously.

Public-key algorithms are most often employed to exchange secret keys securely. The process by which two parties can exchange keys over an unsecure medium is called a key agreement protocol. Common key agreement protocols are the digital envelopes and digital signatures (using the SHA-1 and MD5 hash algorithms).

A limitation of public-key cryptography is that multiple users might share the same set of keys, making it difficult to establish each party's identity. Public Key Infrastructure (PKI) provides a solution by integrating public-key cryptography with digital certificates and certificate authorities to authenticate parties in a transaction.

The Secure Sockets Layer (SSL) protocol is a nonproprietary protocol that secures communication between two computers on the Internet. SSL implements public-key cryptography using the RSA algorithm and digital certificates to authenticate the server in a transaction and to protect private information as it passes over the Internet. SSL transactions do not require client authentication; many servers consider a valid credit card number to be sufficient for authentication in secure purchases.

Virtual Private Networks (VPNs) provide secure communications over public connections. Encryption enables VPNs to provide the same services and security as private networks. A VPN is created by establishing a secure communication channel over the Internet. IPSec (Internet Protocol Security), which uses public-key and symmetric-key cryptography to ensure data integrity, authentication and confidentiality, is commonly used to implement a secure tunnel.

Wireless devices have limited bandwidth and processing power, high latency and unstable connections, so establishing secure wireless communication can be challenging. The Wired Equivalent Privacy (WEP) protocol protects wireless communication by encrypting transmitted data and preventing unauthorized access to the wireless network. WEP has several drawbacks, which make it to be too weak for many environments. Wi-Fi Protected Access (WPA) addresses these issues by providing improved data encryption and by enabling user authentication, a feature not supported by WEP.

A primary advantage to open-source security applications is interoperability—open-source applications tend to implement standards and protocols that many developers include in their products. Another advantage to open-source security is that an application's source code is available for extensive testing and debugging by the community at large. The primary weaknesses of proprietary security are nondisclosure and the fact that the number of collaborative users that can search for security flaws and contribute to the overall security of the application is limited. Proprietary systems, however, can be equally as secure as open-source systems.

The UNIX password file is stored in encrypted form. When a user enters a password, it is encrypted and compared to the password file. Thus, passwords are unrecoverable even by the system administrator.

With the UNIX *setuid* permission feature, a program may be executed by one user using the privileges of another user. This powerful feature has security flaws, particularly when the resulting privilege is that of the "superuser" (who has access to all files in a UNIX system).

Key Terms

3DES-See Triple DES.

Advanced Encryption Standard (AES)—Standard for symmetric encryption that uses Rijndael as the encryption method. AES has replaced the Data Encryption Standard (DES) because AES provides enhanced security.

access control list—List that stores one entry for each access right granted to a subject for an object. An access control list consumes less space than an access control matrix.

access control matrix—Matrix that lists system's subjects in the rows and the objects to which they require access in the columns. Each cell in the matrix specifies the actions that a subject (defined by the row) can perform on an object (defined by the column). Access control matrices typically are not implemented because they are sparsely populated.

access right—Defines how various subjects can access various objects. Subjects may be users, processes, programs or other entities. Objects are information-holding entities; they may be physical objects that correspond to disks processors or main memory or abstract objects that correspond to data structures, processes or services. Subjects are also considered to be objects of the system; one subject may have rights to access another.

air gap technology—Network security solution that complements a firewall. It secures private data from external users accessing the internal network.

antivirus software—Program that attempts to identify, remove and otherwise protect a system from viruses.

application-level gateway—Hardware or software that protects the network against the data contained in packets. If the message contains a virus, the gateway can block it from being sent to the intended receiver.

authentication (secure transaction)—One of the five fundamental requirements for a successful, secure transaction

Authentication deals with how the sender and receiver of a message verify their identities to each other.

authentication header (AH) (IPSec)—Information that verifies the identity of a packet's sender and proves that a packet's data was not modified in transit.

authentication server scripts—Single sign-on implementation that authenticates users via a central server, which establishes connections between the user and the applications the user wishes to access.

authorization (secure transaction) —One of the five fundamental requirements for a successful, secure transaction. Authorization deals with how to manage access to protected resources on the basis of user credentials.

back-door program—Resident virus that allows an attacker complete, undetected access to the victim's computer resources.

biometrics—Technique that uses an individual's physical characteristics, such as fingerprints, eyeball iris scans or face scans, to identify the user.

block cipher—Encryption technique that divides a message into fixed-size groups of bits to which an encryption algorithm is applied.

boot sector virus—Virus that infects the boot sector of the computer's hard disk, allowing it to load with the operating system and take control of the system.

brute-force cracking—Technique to compromise a system simply by attempting all possible passwords or by using every possible decryption key to decrypt a message.

buffer overflow—Attack that sends input that is larger than the space allocated for it. If the input is properly coded and the system's stack is executable, buffer overflows can enable an attacker to execute malicious code.

capability (access control mechanism)—Token that grants a subject privileges for an object. It is analogous to a ticket the bearer may use to gain access to a sporting event.

certificate authority (CA)—Financial institution or other trusted third party, such as VeriSign, that issues digital certificates.

certificate authority hierarchy—Chain of certificate authorities, beginning with the root certificate authority, that authenticates certificates and CAs.

certificate repositories—Locations where digital certificates are stored.

certificate revocation list (CRL)—List of cancelled and revoked certificates. A certificate is cancelled/revoked if a private key is compromised before its expiration date.

cipher—Mathematical algorithm for encrypting messages. Also called cryptosystem.

ciphertext—Encrypted data.

cracker—Malicious individual that is usually interested in breaking into a system to disable services or steal data.

credential—Combination of user identity (e.g., username) and proof of identify (e.g., password).

cryptanalytic attack—Technique that attempts to decrypt ciphertext without possession of the decryption key. The most common such attacks are those in which the encryption algorithm is analyzed to find relations between bits of the encryption key and bits of the ciphertext.

cryptography—Study of encoding and decoding data so that it can be interpreted only by the intended recipients.

cryptosystem—Mathematical algorithm for encrypting messages. Also called a cipher.

Data Encryption Standard (DES)—Symmetric encryption algorithm that use a 56-bit key and encrypts data in 64-bit blocks. For many years, DES was the encryption standard set by the U.S. government and the American National Standards Institute (ANSI). However, due to advances in computing power, DES is no longer considered secure—in the late 1990s, specialized DES cracker machines were built that recovered DES keys after a period of several hours.

denial-of-service (DoS) attack—Attack that prevents a system from properly servicing legitimate requests. In many DoS attacks, unauthorized traffic saturates a network's resources, restricting access for legitimate users. Typically, the attack is performed by flooding servers with data packets.

digital certificate—Digital document that identifies a user or organization and is issued by a certificate authority. A digital certificate includes the name of the subject (the organization or individual being certified), the subject's public key, a serial number (to uniquely identify the certificate), an expiration date, the signature of the trusted certificate authority and any other relevant information.

digital envelope—Technique that protects message privacy by sending a package including a message encrypted using a secret key and the secret key encrypted using public-key encryption.

digital notary service—See time-stamping agency.

digital signature—Electronic equivalent of awritten signature. To create a digital signature, a sender first applies a hash function to the original plaintext message. Next, the sender uses the sender's private key to encrypt the message digest (the hash value). This step creates a digital sig-

nature and validates the sender's identity, because only the owner of that private key could encrypt the message.

Digital Signature Algorithm (DSA)—U.S. government's digital authentication standard.

digital watermark—Popular application of steganography that hides information in unused (or rarely used) portions of a file.

discretionary access control (DAC)—Access control model in which the creator of an object controls the permissions for that object.

distributed denial-of-service attack—Attack that prevents a system from servicing requests properly by initiating packet flooding from separate computers sending traffic in concert.

DNS (domain name system) attack—Attack that modifies the address to which network traffic for a particular Web site is sent. Such attacks can be used to redirect users of a particular Web site to another, potentially malicious Web site.

encapsulating security payload (ESP) (IPSec)—Message data encrypted using symmetric-key ciphers to protect the data from eavesdroppers while the IP packet is transmitted across public communication lines.

Encrypting File System (EFS)—NTFS feature that uses cryptography to protect files and folders in Windows XP Professional and Windows 2000. EFS uses secret-key and public-key encryption to secure files. Each user is assigned a key pair and certificate that are used to ensure that only the user that encrypted the files can access them.

firewall—Software or hardware that protects a local area network from packets sent by malicious users from an external network.

hacker—Experienced programmer who often program as much for personal enjoyment as for the functionality of the application. This term is often used when the term cracker is more appropriate.

heuristic scanning—Antivirus technique that detects viruses by their program behavior.

host-based intrusion detection—IDS that monitors system and application log files.

integrity (secure transaction)—One of the five fundamental requirements for a successful, secure transaction. Integrity deals with how to ensure that the information you send or receive has not been compromised or altered.

Internet Key Exchange (IKE) (IPSec)—Key-exchange protocol used in IPSec to perform key management, which allows secure key exchange.

Internet Protocol Security (IPSec)—Transport-layer security protocol that provides data privacy, integrity and authentication.

intrusion detection system (IDS)—Application that monitors networks and application log files, which record information about system behavior, such as the time at which operating services are requested and the name of the process that requests it.

IP spoofing—Attack in which an attacker simulates the IP address of an authorized user or host to obtain unauthorized access to resources.

Kerberos—Freely available, open-source authentication and access control protocol developed at MIT that provides protection against internal security attacks. It employs secret-key cryptography to authenticate users in a network and to maintain the integrity and privacy of network communications.

key—Input to a cipher to encrypt data; keys are represented by **a** string of digits.

key agreement protocol—Rules that govern key exchange between two parties over an unsecure medium.

key distribution center (KDC)—Central authority that shares a different secret key with every user in the network.

key generation—Creation of encryption keys.

layered biometric verification (LBV)—Authentication technique that uses multiple measurements of human features (such as face, finger and voice prints) to verify a user's identity.

log file—Records information about system behavior, such as the time at which operating services are requested and the name of the process that requests them.

logic bomb—Virus that executes its code when a specified condition is met.

Lucifer algorithm—Encryption algorithm created by Horst Feistel of IBM, which was chosen as the DES by the United States government and the National Security Agency (NSA) in the 1970s.

Macintosh—Apple Computer's PC line that introduced the GUI and mouse to mainstream computer users.

Mac OS—Line of operating systems for Apple Macintosh computers first introduced in 1997.

mandatory access control (MAC)—Access control model in which policies predefine a central permission scheme by which all subjects and objects are controlled.

MD5 Message Digest Algorithm—Hash algorithm developed by Professor Ronald L. Rivest at MIT that is widely used to implement digital signatures.

message digest—Hash value produced by algorithms such as SHA-1 and MD5 when applied to a message.

message integrity—Property indicating whether message has been altered in transmission.

National Institute of Standards and Technology (NIST)—Organization that sets cryptographic (and other) standards for the U.S. government.

network-based intrusion detection—IDS that monitors traffic on a network for any unusual patterns that might indicate DoS attacks or attempted entry into a network by an unauthorized user.

nonrepudiation—Issue that deals with how to prove that a message was sent or received.

Online Certificate Status Protocol (OCSP)-Protocol that validates certificates in real time.

OpenBSD—BSD UNIX system whose primary goal is security.

Operationally Critical Threat, Asset and Vulnerability Evaluation (OCTAVE) method—Technique for evaluating security threats of a system developed at the Software Engineering Institute at Carnegie Mellon University.

Orange Book—Document published by U.S. Department of Defence (DoD) to establish guidelines for evaluating the security features of operating systems.

packet-filtering firewall—Hardware or software that examines all data sent from outside its LAN and rejects data packets based on predefined rules, such as reject packets that have local network addresses or reject packets from certain addresses or ports.

password aging—Technique that attempts to improve security by requiring users to change their passwords periodically.

password protection—Authentication technique that relies on a user's presenting a username and corresponding password to gain access to a resource or system.

password salting—Technique that inserts characters at various positions in the password before encryption to reduce vulnerability to brute-force attacks.

payload—Code inside a logic bomb that is executed when a specified condition is met.

plaintext—Unencrypted data.

Platform for Privacy Preferences (P3P)—Protects the privacy of information submitted to single sign-on and other applications by allowing users to control the personal information that sites collect.

policy creation authority—Organization that sets policies for obtaining digital certificates.

polymorphic virus—Virus that attempts to evade known virus lists by modifying its code (e.g., via encryption, substitution, insertion, and the like) as it spreads.

Pretty Good Privacy (PGP)—Public-key encryption system primarily used to encrypt e-mail messages and files, designed in 1991 by Phillip Zimmermann.

privacy (secure transaction)—One of the five fundamental requirements for a successful, secure transaction. Privacy deals with how to ensure that the information transmitted over the Internet has not been captured or passed to a third party without user knowledge.

private key—Key in public-key cryptography that should be known only by its owner. If its corresponding public key encrypts a message, only the private key should be able to decrypt it.

privilege (access right)—The manner in which a subject can access an object.

protection—Mechanism that prevents unauthorized use of resources such as hardware and operating system services.

protection domain—Collection of access rights. Each access right in a protection domain is represented as an ordered pair with fields for the object name and applicable privileges.

public key—Key in public cryptography that is available to all users that wish to communicate with its owner. If the public key encrypts a message, only the corresponding private key can decrypt it.

public-key cryptography—Asymmetric cryptography technique that employs two inversely related keys: a public key and a private key. To transmit a message securely, the sender uses the receiver's public key to encrypt the message. The receiver then decrypts the message using his or her unique private key.

Public Key Infrastructure (PKI)—Technique that integrates public-key cryptography with digital certificates and certificate authorities to authenticate parties in a transaction.

resident virus—Virus that, once loaded into memory, executes until the computer is powered down.

restricted algorithm—Algorithm that provides security by relying on the sender and receiver to use the same encryption algorithm and maintain its secrecy.

random-scanning algorithm—Algorithm that uses pseudorandom numbers to generate a broad distribution of IP addresses.

real-time scanner—Software that resides in memory and actively prevents viruses.

root key—Used by the Internet Policy Registration Authority (IPRA) to sign certificates exclusively for policy creation authorities.

Rijndael—Block cipher developed by Dr. Joan Daemen and Dr. Vincent Rijmen of Belgium. The algorithm can be implemented on a variety of processors.

role (RBAC) — Represents a set of tasks assigned to a member of an organization. Each role is assigned a set of privileges, which define the objects that users in each role can access.

role-based access control (RBAC)—Access control model in which users are assigned roles.

RSA—Popular public-key algorithm, which was developed in 1977 by MIT professors Ron Rivest, Adi Shamir and Leonard Adleman.

secret-key cryptography—Technique that performs encryption and decryption using the same secret key to encrypt and decrypt a message. The sender encrypts a message using the secret key, then sends the encrypted message to the intended recipient, who decrypts the message using the same secret key. Also called symmetric cryptography.

Secure Hash Algorithm (SHA-1)—Popular hash function used to create digital signatures; developed by NIST

Secure Sockets Layer (SSL)—Nonproprietary protocol developed by Netscape Communications that secures communication between two computers on the Internet.

Security mechanism—Method by which the system implements its security policy. In many systems, the policy changes over time, but the mechanism remains unchanged.

Security model—Entity that defines a system's subjects, objects and privileges.

Security patch—Code that addresses a security flaw.

Security policy—Rules that govern access to system resources.

Service ticket (Kerberos)—Ticket that authorizes the client's access to specific network services.

Session key—Secret key that is used for the duration of a transaction (e.g., a customer's buying merchandise from an online store).

Shadow password file (UNIX)—Protects the password file from crackers by storing information other than the encrypted passwords in the normal password file and storing the encrypted passwords in the shadow password file that can be accessed only by users with root privileges.

Signature-scanning virus detection—Antivirus technique that relies on knowledge of virus code.

Single sign-on— Simplifies the authentication process by allowing the user to log in once, using a single password.

smart card—Credit card size data store that serves many functions, including authentication and data storage.

steganography—Technique that hides information within other information, derived from Latin roots meaning "covered writing."

substitution cipher—Encryption technique whereby every occurrence of a given letter is replaced by a different letter. For example, if every "a" were replaced by a "b,' every "b" by a "c," and so on the word "security" would encrypt to "tfdvsjuz."

static analysis—Intrusion detection method which attempts to detect when applications have been corrupted by a hacker.

system penetration—Successful breach of computer security by an unauthorized external user.

symmetric cryptography—See secret-key cryptography.

Ticket Granting Service (TGS) (Kerberos)—Server that authenticates client's rights to access specific network services.

Ticket-Granting Ticket (TGT) (Kerberos)—Ticket returned by Kerberos's authentication server. It is encrypted with the client's secret key that is shared with the authentication server. The client sends the decrypted TGT to the Ticket Granting Service to request a service ticket.

time bomb—Virus that is activated when the clock on the computer matches a certain time or date.

timestamping (nonrepudiation)—Technique that binds a time and date to a digital document, which helps solve the problem of nonrepudiation.

time-stamping agency—Organization that digitally time stamps a document that has been digitally signed.

token (in token-based authentication)—Unique identifier for authentication.

token-based authentication—Authentication technique that issues a token unique to each session, enabling users to access specific applications.

transient virus—Virus that attaches itself to a particular computer program. The virus is activated when the program is run and deactivated when the program is terminated.

transposition cipher—Encryption technique whereby the ordering of the letters is shifted. For example, if every other letter, starting with "s," in the word "security" creates the first word in the ciphertext and the remaining letters create the second word in the ciphertext, the word "security" would encrypt to "scrt euiy."

Triple DES—Variant of DES that can be thought of as three DES systems in series, each with a different secret key that operates on a block. Also called 3DES.

Trojan horse—Malicious program that hides within a trusted program or simulates the identity of a legitimate program or feature, while actually causing damage to the computer or network when the program is executed.

two-factor authentication—Authentication technique that employs two means to authenticate the user, such as biometrics or a smart card used in combination with a password.

variant—Virus whose code has been modified from its original form, yet still retains its malicious payload.

Virtual Private Network (VPN)—Technique that securely connects remote users to a private network using public communication lines. VPNs are often implemented using IPSec.

virus—Executable code (often sent as an attachment to an e-mail message or hidden in files such as audio clips, video clips and games) that attaches to or overwrites other files to replicate itself, often harming the system on which it resides.

virus signature—Segment of code that does not vary between virus generations.

Web defacing—Attack that maliciously modifies an organization's Web site.

Wi-Fi Protected Access (WPA)—Wireless security protocol intended to replace WEP by providing improved data encryption and by enabling user authentication.

Wired Equivalent Privacy (WEP)—Wireless security protocol that encrypts transmitted data and prevents unauthorized access to the wireless network.

workstation login scripts—Simple form of single sign-on in which users log in at their workstations, then choose applications from a menu.

worm—Executable code that spreads and infects files over a network. Worms rarely require any user action to propagate, nor do they need to be attached to another program or file to spread.

Exercises

19.1 Why is a precise statement of security requirements critical to the determination of whether a given system is secure?

19.2 Sharing and protection are conflicting goals. Give three significant examples of sharing supported by operating systems. For each, explain what protection mechanisms are necessary to control the sharing.

19.3 Give several reasons why simple password protection is the most common authentication scheme in use today. Discuss the weaknesses inherent in password protection schemes.

19.4 An operating systems security expert has proposed the following economical way to implement a reasonable level of security: Simply tell everyone who works at an installation that the operating system contains the latest and greatest in security mechanisms. Do not actually build the mechanisms, the expert says, just tell everyone that the mechanisms are in place. Who might such a scheme deter from attempting a security violation? Who would not likely be deterred? Discuss the advantages and disadvantages of the scheme. In what types of installations might this scheme be useful? Suggest a simple modification to the scheme that would make it far more effective, yet still far more economical than a comprehensive security program.

19.5 What is the principle of least privilege? Give several reasons why it makes sense. Why is it necessary to keep protection domains small to effect the least-privilege approach to security? Why are capabilities particularly useful in achieving small protection domains?

19.6 How does a capabilities list differ from an access control list?

19.7 Why is an understanding of cryptography important to operating systems designers? List several areas within operating systems in which the use of cryptography would greatly improve security. Why might a designer choose not to include encryption in all facets of the operating system?

19.8 Why is it desirable to incorporate certain operating systems security functions directly into hardware? Why is it useful to microprogram certain security functions?

19.9 Give brief definitions of each of the following terms.

a. cryptography
b. Data Encryption Standard (DES)
c. privacy problem
d. integrity problem
e. problem of nonrepudiation
f. plaintext
g. ciphertext
h. unsecure channel
i. eavesdropper
j. cryptanalysis

19.10 Give brief definitions of each of the following terms.

a. public-key system
b. public key
c. private key
d. digital signature

19.11 Why are denial-of-service attacks of such great concern to operating systems designers? List a few types of denial-of-service attacks. Why is it difficult to detect a distributed denial-of-service attack on a server?

19.12 What is a log file? What information might an operating system deposit in a log file? How does a log act as a deterrent to those who would commit a security violation?

19.13 Explain how public-key cryptography systems provide an effective means of implementing digital signatures.

19.14 Explain how cryptography is useful in each of the following.

a. protecting a system's master list of passwords
b. protecting stored files
c. protecting vulnerable transmissions in computer networks

19.15 The UNIX systems administrator cannot determine the password of a user who has lost his or her password. Why? How can that user regain access to the computer?

19.16 How could a UNIX system user seize control of a system, thus locking out all other users? What defense may be employed against this technique?

19.17 Why do antivirus software companies and high-security installations hire convicted hackers?

Suggested Projects

19.18 At the moment, it appears that our computing systems are easy targets for penetration. Enumerate the kinds of weaknesses that are prevalent in today's systems. Suggest how to correct these weaknesses.

19.19 Design a penetration study for a large computer system with which you are familiar. Adhere to the following:

a. Perform the study only with the full cooperation of the computer center administrators and personnel.
b. Arrange to have ready access to all available system documentation and software source listings.
c. Agree that any flaws you discover will not be made public until they have been corrected.
d. Prepare a detailed report summarizing your findings and suggesting how to correct the flaws.
e. Your primary goals are to achieve
    i. total system penetration
    ii. denial of access to legitimate system users
    iii. the crashing of the system.
f. You may access the system only through conventional mechanisms available to nonprivileged users.
g. Your attacks must be software and/or firmware oriented.

19.20 Prepare a research paper on the RSA algorithm. What mathematical principles aid the RSA algorithm in being difficult to crack?

19.21 Prepare a research paper on how antivirus software provides security while being minimally invasive.

19.22 Prepare a research paper on the implementation of PGP Explain why the name "Pretty Good Privacy" is appropriate.

19.23 Prepare a research paper on the most costly viruses and worms. How did these worms or viruses spread? What flaws did they exploit?

19.24 Prepare a research paper on how IP spoofing is performed.

19.25 Discuss the ethical and legal issues surrounding the practice of hacking.

Suggested Simulation

19.26 Create an application to encode and decode secret messages within bitmap pictures. When encoding, modify the least significant bit in each byte to store information about the message. To decode, just reassemble the message using the least significant bit of each byte.

Recommended Reading

Computer security has been researched extensively in academic and commercial environments. Most topics in the field are constantly evolving. Although the problems of cryptography and access control are well characterized, techniques that

address security threats (such as software exploitation and network attacks) must continually evolve as new hardware, software and protocols with different vulnerabilities emerge.

A survey of cryptography and the related problem of computability is provided in Jorg Rothe's "Some Facets of Complexity Theory and Cryptography: A Five-Lecture Tutorial."[151] The problem of access control is actively discussed in the literature; a survey of access control can be found in Sandhu and Samarati's "Access Control: Principles and Practice."[152]

Due to the importance of security to e-business and the Internet revolution, a large number of security trade journals and publications have emerged since the 1990s. Commercial trade journals include *Information Security, SC (Secure Computing) Magazine* and *Access Control and Security Systems.* Academic security journals include the *Journal of Computer Security* and the *ACM Transactions on Information and System Security (TISSEC).* Dieter Gollmann provides a thorough discussion of computer security theory.[153] The bibliography for this chapter is located on our Web site at www.deitel.com/books/os3e/Bibliography.pdf.

Works Cited

1. Gates, B., "Bill Gates: Trustworthy Computing," January 17,2002, <www.wired.com/news/business/0,1367,49826,00.html>.
2. McFarland, M., "Ethical Issues of Computer Use," Syllabus for course Mc690, Boston College, Chestnut Hill, MA, Spring 1989.
3. Parnas, D. L., "Software Aspects of Strategic Defense Systems," *Communications of the ACM,* Vol. 28, No. 12, December 1985, pp. 1326-1335.
4. "Cryptography—Caesar Cipher," <http://www.trincoll.edu/depts/cpsc/cryptography/caesar.html>.
5. "Methods of Transposition," <http://home.ecn.ab.ca/~jsavard/crypto/pp0102.htm>.
6. RSA Laboratories, "RSA Laboratories' Frequently Asked Questions About Today's Cryptography," Version 4.1, 2000, RSA Security Inc., <www.rsasecurity.com/rsalabs/faq>.
7. Cherowitzo, B., "Math 5410 Data Encryption Standard (DES)," February 6, 2002, <www-math.cudenver.edu/~wcherowi/courses/m5410/m5410des.html>.
8. Dworkin, M., "Advanced Encryption Standard (AES) Fact Sheet," March 5, 2001, <csrc.nist.gov/CryptoToolkit/aes/round2/aesfact.html>.
9. Rijmen, V., "The Rijndael Page," May 20, 2003, <www.esat.kuleuven.ac.be/~rijmen/rijndael>-
ro. McNett, D., "RC5-64 HAS BEEN SOLVED!" September 27, 2002, <www.distributed.net/pressroom/news-20020926.html>.
11. RSA Laboratories, "RSA-Based Cryptographic Schemes," 2003, <www.rsasecurity.com/rsalabs/rsa_algorithm>.
12. Ytteborg, S. S., "Overview of PGP," <www.pgpi.org/doc/overview>.
13. RSA Security Inc., "RSA Security at a Glance," 2003, <www.rsasecurity.com/company/corporate.html>.
14. Rivest, R. L.; A. Shamir and L. Adleman, "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems," *Communications of the ACM,* Vol. 21, No. 2, February 1978, pp. 120-126.
15. Bass, T. A., "Gene Genie," *Wired,* August 1995, <www.thomas-bass.com/work12.htm>.
16. Krieger, D., and W. E. Miles, "Innerview: Leonard Adleman," *Networker,* Vol. 7, No. 1, September 1996, <www.usc.edu/isd/publications/networker/96-97/Sep_Oct_96/innerview-adleman .html>.
17. ACM, "ACM: A. M. Turing Award," April 14, 2003, <www.acm.org/announcements/turing_2002.html>.
18. RSA Security Inc., "The Authentication Scorecard," 2003, <www.rsasecurity.com/products/authentication/whitepapers/ASC_WP_0403.pdf>.
19. Lamport, L., "Password Authentication with Insecure Communication," *Communications of the ACM,* Vol. 24, No. 11, November 1981, pp. 770-772.
20. National Infrastructure Protection Center, "Password Protection 101," May 9, 2002, <www.nipc.gov/publications/nipcpub/password.htm>.
21. Deckmyn, D., "Companies Push New Approaches to Authentication," *Computerworld,* May 15,2000, p. 6.
22. Keyware, "Keyware Launches Its New Biometric and Centralized Authentication Products," January 2001, <hwww.keyware.com/press/press.asp?pid=44&menu=4>.
23. Vijayan, X, "Biometrics Meet Wireless Internet," *Computerworld,* July 17,2000, p. 14.
24. Smart Card Forum, "What's So Smart About Smart Cards?" 2003, <www.gemplus.com/basics/download/smartcardforum.pdf>.
25. Gaudin, S., "The Enemy Within," *Network World,* May 8, 2000, pp. 122-126.
26. Needham, R. M. and M. D. Schroeder, "Using Encryption for Authentication in Large Networks of Computers," *Communications of the ACM,* Vol. 21, No. 12, December 1978, pp.993- 999.
27. Steiner, X; C. Neuman and X Schiller, "Kerberos: An authentication service for open network systems," *In Proceedings of the Winter 1988 USENIX,* February 1988, pp. 191-202.
28. Steiner, X; C. Neuman and J. Schiller, "Kerberos: An authentication service for open network systems," *In Proceedings of the Winter 1988 USENIX,* February 1988, pp. 191-202.

29. Trickey, F., "Secure Single Sign-On: Fantasy or Reality," Computer Security Institute, 1997.
30. Musthaler, L., "The Holy Grail of Single Sign-On," *Network World,* January 28,2002, p. 47.
31. Press Release, "Novell Takes the Market Leader in Single Sign-On and Makes It Better," <www.noven.com/news/press/archive/20O3/09/pr03059.html>.
32. "Protection and Security," <http://cs.nmu.edu/~randy/C1asses/CS426/protection_and_security.html>.
33. Lampson, B., "Protection," *ACM Operating Systems Review 8,* January 1,1974, pp. 18-24.
34. Sandhu, R., and E. Coyne, "Role-Based Access Control Models," *IEEE Computer,* February 1996, p. 39.
35. Sandhu, R., and E. Coyne, "Role-Based Access Control Models," *IEEE Computer,* February 1996, p. 40.
36. "Handbook of Information Security Management: Access Control," <www.cccure.org/Documents/HISM/094-099.html>.
37. Chander, A.; D Dean; and J. C. Mitchell, "A State-Transition Model of Trust Management and Access Control," *IEEE Computer, in Proceedings of the 14th Computer Security Foundations Workshop (CSFW),* June 2001, pp. 27-43.
38. Lampson, B., "Protection," *Proceedings of the Fifth Princeton Symposium on Information Sciences and Systems,* 1971, pp. 437-443.
39. Linden, T A., "Operating System Structures to Support Security and Reliable Software," ACM *Computing Surveys (CSUR),* Vol. 8, No. 6, December 1976.
40. Moore, D., et al., "The Spread of the Sapphire/Slammer Worm," *CAIDA,* February 3, 2003, <www.caida.org/outreach/papers/2003/sapphire/>.
41. "Securing B2B," *Global Technology Business,* July 2000, pp. 50-51.
42. Computer Security Institute, "Cyber Crime Bleeds U.S. Corporations, Survey Shows; Financial Losses from Attacks Climb for Third Year in a Row," April 7, 2002, <www.gocsi.com/press/20020407.html>.
43. Connolly, P., "IT Outlook Appears Gloomy," *InfoWorld,* November 16, 2001, <www.infoworld.com/articles/tc/xml/01/ll/19/011119tcstate.xrrfl>.
44. Bridis, T, "U.S. Archive of Hacker Attacks to Close Because It Is Too Busy," *The Wall Street Journal,* May 24,2001, p. BIO.
45. Andress, M., "Securing the Back End," *InfoWorld,* April 5, 2002, <www.infoworld.com/articles/ne/xml/02/04/08/020408neappdetective.xml>.
46. Marshland, R., "Hidden Cost of Technology," *Financial Times,* June 2,2000, p. 5.
47. Spangler, T, "Home Is Where the Hack Is," *Inter@ctive Week,* April 10,2000, pp. 28-34.
48. Whale Communications, "Air Gap Architecture," 2003, <www.whalecommunications.com/site/Whale/Corporate/Whale.asp?pi=264>.
49. Azim, O., and P. Kolwalkar, "Network Intrusion Monitoring," *Business Security Advisor,* March/April 2001, pp. 16-19, <advisor.com/doc/07390>.
50. Wagner, D., and D. Dean, "Intrusion Detection via Static Analysis," *IEEE Symposium on Security and Privacy,* May 2001.
51. Alberts, C, and A. Dorofee, "OCTAVE Information Security Risk Evaluation," January 30, 2001, <www.cert.org/octave/methodintro.html>.
52. Delio, M, "Find the Cost of (Virus) Freedom," *Wired News,* January 14, 2002, <www.wired.com/news/infostructure/0,1377,49681,00.html>.
53. Bridwell, L. M., and P. Tippett, "ICSA Labs Seventh Annual Computer Virus Prevalence Survey 2001," *ICSA Labs,* 2001.
54. Helenius, M., "A System to Support the Analysis of Antivirus Products' Virus Detection Capabilities," *Academic dissertation. Dept. of Computer and Information Sciences, University of Tampere, Finland,* May 2002.
55. Solomon, A., and T. Kay, *Dr. Solomon's PC Anti-virus Book.* Butterworth-Heinemann, 1994, pp. 12-18.
56. University of Reading, "A Guide to Microsoft Windows XP." October 24, 2003, <www.rdg.ac.uk/ITS/info/training/notes/windows/guide/>.
57. Helenius, M., "A System to Support the Analysis of Antivirus Products' Virus Detection Capabilities," *Academic dissertation. Dept. of Computer and Information Sciences, University of Tampere, Finland,* May 2002.
58. OpenBSD Team, "OpenBSD Security," September 17, 2003. <www.openbsd.org/security.html>.
59. Microsoft Corporation, "Top 10 Reasons to Get Windows XP for Home PC Security," April 16, 2003, <www.microsoft.com/WindowsXP/security/top10.asp>.
60. Microsoft Corporation, "What's New in Security for Windov>> XP Professional and Windows XP Home Edition," July 2001. <www.microsoft.com/wi ndowsxp/pro/techinfo/planning/security/whatsnew/WindowsXPSecurity.doc>.
61. *Security Electronics Magazine,* "OpenBSD: Secure by Default." January 2002, <www.semweb.com/jan02/itsecurityjan.htm>.
62. Howard, X, "Daemon News: The BSD Family Tree," April 2001. <www.daemonnews.org/200104/bsd_family.html>.
63. *Security Electronics Magazine,* "OpenBSD: Secure by Default." January 2002, <www.semweb.com/jan02/itsecurityjan.htm>.
64. Jorm, D, "An Overview of OpenBSD Security," August 8, 2000. <www.onlamp.com/pub/a/bsd/2000/08/08/OpenBSD.html>.
65. Howard, J., "Daemon News: The BSD Family Tree," April 2001. <www.daemonnews.org/200104/bsd_fami1y.html>.
66. Howard, X, "Daemon News: The BSD Family Tree," April 2001. <www.daemonnews.org/200104/bsd_family.html>.
67. *Security Electronics Magazine,* "OpenBSD: Secure by Default." January 2002, <www.semweb.com/jan02/itsecurityjan.htrn>.
68. OpenBSD Team, "OpenBSD Security," September 17, 2003. <www.openbsd.org/security.html>.
69. *Security Electronics Magazine,* "OpenBSD: Secure by Default." January 2002, <www.semweb.com/jan02/itsecurityjan.htm>.
70. Jorm, D., "An Overview of OpenBSD Security," August 8, 2000. <www.onlamp.com/pub/a/bsd/2000/08/08/OpenBSD.html>.

71. Howard, X, "Daemon News: The BSD Family Tree," April 2001, <www.daemonnews.org/200104/bsd_family.html>.
72. OpenBSD Team, "Cryptography in OpenBSD," September 25, 2003, <www.openbsd.org/crypto.html>.
73. OpenBSD Team, "OpenBSD Security," September 17, 2003, <www.openbsd.org/security.html>.
74. Howard, X, "Daemon News: The BSD Family Tree," April 2001, <www.daemonnews.org/200104/bsd_family.html>.
75. OpenBSD Team, "OpenBSD Security," September 17, 2003, <www.openbsd.org/security.html>.
76. Mullen, T, "Windows Server 2003 - Secure by Default," April 27, 2003, <www.securityfocus.com/columnists/157>.
77. OpenBSD Team, "OpenBSD," September 27, 2003, <www.open¬ bsd.org>.
78. *Security Electronics Magazine,* "OpenBSD: Secure by Default," January 2002, <www.semweb.com/jan02/itsecurityjan.htm>.
79. Sanford, G., "Lisa/Lisa 2/Mac XL," <www.apple-history.com/ noframes/body.php?page=gallery&model=lisa>.
80. Sanford, G., "Macintosh 128k," <www.apple-history.com/nof- rames/body.php?page=gallery&model=128k>.
81. Trotot, X, "Mac™ OS History," <perso.club-internet.fr/ jctrotot/Perso/History.html>.
82. Horn, B., "On Xerox, Apple, and Progress," <www.apple-his¬ tory.com/noframes/body.php?page=gui_hornl>.
83. Horn, B., "On Xerox, Apple, and Progress," <www.apple-his¬ tory.com/noframes/body.php?page=gui _hornl>.
84. Sanford, G, "Lisa/Lisa 2/Mac XL," <www.apple-history.com/ noframes/body.php?page=gallery&model=lisa>.
85. Apple Computer, "1984 Commercial," <www.apple-his- tory.com/noframes/body.php?page=gallery&model=1984>.
86. Sanford, G., "Macintosh 128k," <www.apple-history.com/nof- rames/body.php?page=gallery&model=128k>.
87. Sanford, G, "Company History: 1985-1993," <www.apple-his¬ tory.com/noframes/body.php?page=hi story&section=h4>.
88. University of Utah, "Mac OS History," September 23, 2003, <www.macos.Utah.edu/Documentation/MacOSXClasses/ macosxone/maci ntosh.html>.
89. University of Utah, "Mac OS History," September 23, 2003, <www.macos.Utah.edu/Documentation/MacOSXClasses/ macosxone/macintosh.html>.
90. Trotot, X, "Mac™ OS History," <perso.club-internet.fr/ jctrotot/Perso/History.html>.
91. University of Utah, "Mac OS History," September 23, 2003, <www.macos.Utah.edu/Documentation/MacOSXClasses/ macosxone/macintosh.html>.
92. Apple Computer, Inc., "The Evolution of Darwin," 2003, <developer.apple.com/darwin/history.html>.
93. Apple Computer, Inc., "Apple—Mac OS X—Features—UNIX," 2003, <www.apple.com/macosx/features/unix/>.
94. Apple Computer, Inc., "Apple—Mac OS X—Features —Dar- win," 2003,<www.apple.com/macosx/features/darwin/>.
95. Apple Computer, Inc., "Apple—Mac OS X—Features—Secu- rity," 2003,<www.apple.com/macosx/features/security/>.
96. Apple Computer, Inc., "Apple—Mac OS X—Features—Win- dows," 2003,<www.apple.com/macosx/features/windows/>.
97. Dynamoo, "Orange Book FAQ," June 2002, <www.dynamoo.com/ orange/faq.htm>.
98. The Jargon Dictionary, "Orange Book," <info.astrian.net/ jargon/terms/o/Orange_Book.html>.
99. Dynamoo, "Orange Book Summary," March 2003, <www.dynamoo.com/orange/summary.htm>.
100. Dynamoo, "Orange Book—Full Text," June 2002, <www.dynamoo.com/orange/fulltext.htm>.
101. "EPL Entry CSC-EPL-95/006.D," January 2000, <www.radium. ncsc.mil/tpep/epl/entries/CSC-EPL-95-006-D.html>.
102. "EPL Entry CSC-EPL-93/008.A," July 1998, <www.radium. ncsc.mil/tpep/epl/entries/CSC-EPL-93-008-A.html>.
103. "EPL Entry CSC-EPL-93/003.A," July 1998, <www.radium. ncsc.mil/tpep/epl/entries/CSC-EPL-93-0O3-A.html>.
104. "EPL Entry CSC-EPL-93/006.A," July 1998, <www.radium. ncsc.mil/tpep/epl/entries/CSC-EPL-93-006-A.html>.
105. Corbato, F. J., and V. A. Vyssotsky, "Introduction and Overview of the Multics System," <www.multicians.org/fjccl.html >.
106. "EPL Entry CSC-EPL-90/001.A," July 1998, <www.radium. ncsc.mil/tpep/epl/entries/CSC-EPL-90-001-A.html>.
107. "EPL Entry CSC-EPL-92/003.E," June 2000, <www.radium, ncsc.mil/tpep/epl/entries/CSC-EPL-92-003-E.html>.
108. "EPL Entry CSC-EPL-94/006," July 1998, <www.radium. ncsc.mil/tpep/epl/entries/CSC-EPL-94-006.html>.
109. RSA Laboratories, "4.1.3.14 How Are Certifying Authorities Susceptible to Attack?" April 16, 2003, <www.rsasecurity.com/ rsalabs/faq/4-1-3-14.html>.

no. Hulme, G., "VeriSign Gave Microsoft Certificates to Imposter," *Information Week,* March 3,2001.

in. Ellison, C, and B. Schneier, "Ten Risks of PKI: What You're Not Being Told about Public Key Infrastructure," *Computer Security Journal, 2000.*

112. "X.509 Internet Public Key Infrastructure Online Certificate Sta- tus Protocol-OCSP," RFC 2560, June 1999, <www.ietf.org/ rfc/rfc2560.txt>.
113. Abbot, S., "The Debate for Secure E-Commerce," *Performance Computing,* February 1999, pp. 37-42.
114. Wilson, T, "E-Biz Bucks Lost Under the SSL Train," *Internet* Week, May 24,1999, pp. 1,3.
115. Gilbert, H., "Introduction to TCP/IP," February 2, 1995, <www.yale.edu/pclt/COMM/TCPIP.HTM>.
116. RSA Laboratories, "Security Protocols Overview," 1999, <www.rsasecurity.com/standards/protocols>.
117. "The TLS Protocol Version 1.0," RFC 2246, January 1999, <www.ietf.org/rfc/rfc2246.txt>.
118. Cisco Systems, Inc., "VPN Solutions," 2003, <www.cisco.com/ warp/public/44/solutions/network/vpn.shtml>.

119. Burnett, S., and S. Paine, *RSA Security's Official Guide to Cryptography,* Berkeley: Osborne McGraw-Hill, 2001, p. 210.

120. RSA Laboratories, "3.6.1 What Is Diffie-Hellman?" April 16, 2003, <www.rsasecurity.com/rsalabs/faq/3-6-1.html>.

121. Naik, D., *Internet Standards and Protocols,* Microsoft Press, 1998, pp. 79-80.

122. Grayson, M., "End the PDA Security Dilemma," *Communication News,* February 2001, pp. 38-40.

123. Fratto, M., "Tutorial: Wireless Security," January 22, 2001, <www.networkcomputing.com/1202/1202fldl.html>.

124. Westoby, K., "Security Issues Surrounding Wired Equivalent Privacy," March 26, 2002, <www.cas.monaster.ca/~wmfarmer/SE-4C03-02/projects/student_work/westobkj.html>.

125. Borisov, R; I. Goldberg; and D. Wagner, "Security of the WEP algorithm," <www.isaac.cs.berkeley.edu/isaac/wep-faq.html>.

126. Geier, X, "802.11 WEP: Concepts and Vulnerability," June 20, 2002, <www.wi-fiplanet.com/tutorials/article.php/1368661>.

127. Geier, J., "WPA Plugs Holes in WEP," March 31, 2003, <www.nwfusion.com/research/2003/0331wpa.html>.

128. CNN Wireless Society, "Special Report: Wireless 101," 2003, <www.cnn.com/SPECIALS/2003/wireless/interactive/wi reless101/index. htmlx

129. Katzenbeisser, S., and F. Petitcolas, *Information Hiding: Techniques for Steganography and Digital Watermarking,* Norwood: Artech House, Inc., 2000, pp. 1-2.

130. Evers, J., "Worm Exploits Apache," *InfoWorld,* July 1, 2002, <www.infoworld.com/articles/hn/xml/02/07/01/020701hnapache.xml>.

131. Lasser, X, "Irresponsible Disclosure," *SecurityFocus Online,* June 26,2002, <online.securityfocus.com/columnists/91>.

132. Grampp, F. X, and R. H. Morris, "UNIX Operating System Security," *AT&T Bell Laboratories Technical Journal,* Vol. 63, No. 8, October 1984, pp. 1649-1672.

133. Wood, P., and S. Kochan, *UNIX System Security,* Hasbrouck Heights, NJ: Hayden Book Co., 1985.

134. Farrow, R., "Security Issues and Strategies for Users," *UNIX World,* April 1986, pp. 65-71.

135. Farrow, R., "Security for Superusers, or How to Break the UNIX System," *UNIX World,* May 1986, pp. 65-70.

136. Filipski, A., and J. Hanko, "Making UNIX Secure," *Byte,* April 1986, pp. 113-128.

137. Coffin, S., *UNIX: The Complete Reference,* Berkeley, CA: Osborne McGraw-Hill, 1988.

138. Hecht, M. S.; A. Johri; R. Aditham; and T. J. Wei, "Experience Adding C2 Security Features to UNIX," *USENIX Conference Proceedings,* San Francisco, June 20-24,1988, pp. 133-146.

139. Filipski, A., and J. Hanko, "Making UNIX Secure," *Byte,* April 1986, pp. 113-128.

140. Linux Password & Shadow File Formats, <www.tldp.org/LDP/lame/LAME/linux-admin-made-easy/shadow-file-formats.html>.

141. Coffin, S., *UNIX: The Complete Reference,* Berkeley, CA: Osborne McGraw-Hill, 1988.

142. Kramer, S. M., "Retaining SUID Programs in a Secure UNIX." *USENIX Conference Proceedings,* San Francisco, June 20-24. 1988, pp. 107-118.

143. Farrow, R., "Security Issues and Strategies for Users," *UNIX World,* April 1986, pp. 65-71.

144. Filipski, A., and J. Hanko, "Making UNIX Secure," *Byte,* April 1986, pp. 113-128.

145. Farrow, R., "Security Issues and Strategies for Users," *UNIX World,* April 1986, pp. 65-71.

146. Grampp, F. T., and R. H. Morris, "UNIX Operating System Security," *AT&T Bell Laboratories Technical Journal,* Vol. 63, No. 8. October 1984. pp. 1649-1672.

147. Coffin. S., *UNIX: The Complete Reference,* Berkeley, CA: Osborne McGraw-Hill, 1988.

148. Bauer, D. S., and M. E. Koblentz, "NIDX-A Real-Time Intrusion Detection Expert System," *USENIX Conference Proceedings,* San Francisco, June 20-24,1988, pp. 261-274.

149. Farrow, R., "Security Issues and Strategies for Users," *UNIX World,* April 1986. pp. 65-71.

150. Filipski, A., and J. Hanko, "Making UNIX Secure," *Byte,* April 1986, pp. 113-128.

151. Rothe, J, "Some Facets of Complexity Theory and Cryptography: A Five-Lecture Tutorial," *ACM Computing Surveys (CSUR),Vol* 24, No. 4, December 2002.

152. Sandhu, R. S., and P. Samarati, "Access Control: Principles and Practice," *IEEE Communications,* Vol. 32, No. 9,1994, pp. 40-48.

153. Gollmann, D.. *Computer Security,* Hoboken, NJ: John Wiley & Sons Ltd. 1999.

Operating System Case Studies

Our children may learn about the heroes of the past.
Our task is to make ourselves architects of the future.
—Mzee Jomo Kenyatta —

Part 8

The final two chapters present in-depth case studies of Linux and Microsoft Windows XP These case studies reinforce the text's key concepts and demonstrate how operating system principles are applied in real-world operating systems. Chapter 20 examines the history and core components of Linux 2.6—the most popular open-source operating system— including kernel architecture, process management, memory management, file systems, I/O management, synchronization, IPC, networking, scalability and security. Chapter 21 explores the internals of Windows XP—the most popular proprietary operating system. The case study examines XP's history, design goals and core components, including system architecture, system management mechanisms, process and thread management, memory management, file systems management, input/output management, IPC, networking, scalability and security.

Freely ye have received, freely give.
-Matthew 10:8-

Freely ye have received, freely give.

-Matthew 10:8-

The world is moving so fast these days that the man who says it can't be done is generally interrupted by someone doing it.

—Elbert Hubbard—

When your Daemon is in charge, do not try to think consciously. Drift, wait and obey.

— Rudyard Kipling—

I long to accomplish a great and noble task, but it is my chief duty to accomplish small tasks as if they were great and noble.

—Helen Keller—

Our children may learn about heroes of the past, Our task is to make ourselves architects of the future.

—Jomo Mzee Kenyatta—

Chapter 20

Case Study: Linux

Objectives

After reading this chapter, you should understand:

- *Linux kernel architecture.*
- *the Linux implementation of operating system components such as process, memory and file management.*
- *the software layers that compose the Linux kernel.*
- *how Linux organizes and manages system devices.*
- *how Linux manages I/O operations.*
- *interprocess communication and synchronization mechanisms in Linux.*
- *how Linux scales to multiprocessor and embedded systems.*
- *Linux security features.*

Chapter Outline

Web Resources | Key Terms | Exercises | Recommended Reading | Works Cited

20.1 Introduction

The Linux kernel version 2.6 is the core of the most popular open-source, freely distributed, full-featured operating system. Unlike that of proprietary operating systems, Linux source code is available to the public for examination and modification and is free to download and install. As a result, users of the operating system benefit from a community of developers actively debugging and improving the kernel, an absence of licensing fees and restrictions and the ability to completely customize the operating system to meet specific needs. Though Linux is not centrally produced by a corporation, Linux users can receive technical support for a fee from Linux vendors or for free through a community of users.

The Linux operating system, which is developed by a loosely organized team of volunteers, is popular in high-end servers, desktop computers and embedded systems. Besides providing core operating system features, such as process scheduling, memory management, device management and file system management, Linux supports many advanced features such as symmetric multiprocessing (SMP), non-uniform memory access (NUMA), access to multiple file systems and support for a broad spectrum of hardware architectures. This case study offers the reader an opportunity to evaluate a real operating system in substantial detail in the context of the operating system concepts discussed throughout this book.

20.2 History

In 1991, Linus Torvalds, a 21-year-old student at the University of Helsinki, Finland, began developing the Linux (the name is derived from "Linus" and "UNIX") kernel as a hobby. Torvalds wished to improve upon the design of Minix, an educational operating system created by Professor Andrew S. Tanenbaum of the Vrije Universiteit in Amsterdam. The Minix source code, which served as a starting point for Torvalds's Linux project, was publicly available for professors to demonstrate basic operating system implementation concepts to their students.[1]

In the early stages of development, Torvalds sought advice about the shortcomings of Minix from those familiar with it. He designed Linux based on these suggestions and made further efforts to involve the operating systems community in his project. In September of 1991, Torvalds released the first version (0.01) of the Linux operating system, announcing the availability of his source code to a Minix newsgroup.[2]

The response led to the creation of a community that has continued to develop and support Linux. Developers downloaded, tested, and modified the Linux code, submitting bug fixes and feedback to Torvalds, who reviewed them and applied the improvements to the code. In October, 1991, Torvalds released version 0.02 of the Linux operating system.[3]

Although early Linux kernels lacked many features implemented in well-established operating systems such as UNIX, developers continued to support the concept of a new, freely available operating system. As Linux's popularity grew.

developers worked to remedy its shortcomings, such as the absence of a login mechanism and its dependence on Minix to compile. Other missing features were floppy disk support and a virtual memory system.[4] Torvalds continued to maintain the Linux source code, applying changes as he saw fit.

As Linux evolved and drew more support from developers, Torvalds recognized its potential to become more than a hobby operating system. He decided that Linux should conform to the POSIX specification to enhance its interoperability with other UNIX-like systems. Recall that POSIX, the Portable Operating System Interface, defines standards for application interfaces to operating system services, as discussed in Section 2.7, Application Programming Interfaces (APIs).[5]

The 1994 release of Linux version 1.0 included many features commonly found in a mature operating system, such as multiprogramming, virtual memory, demand loading and TCP/IP networking.[6] It provided the functionality necessary for Linux to become a viable alternative to the licensed UNIX operating system.

Though it benefited from free licensing, Linux suffered from a complex installation and configuration process. To allow users unfamiliar with the details of Linux to conveniently install and use the operating system, academic institutions, such as the University of Manchester and Texas A&M University, and organizations such as Slackware Linux (www.slackware.org), created Linux **distributions,** which included software such as the Linux kernel, system applications (e.g., user account management, network management and security tools), user applications (e.g., GUIS, Web browsers, text editors, e-mail applications, databases, and games) and tools to simplify the installation process.[7]

As kernel development progressed, the project adopted a version numbering scheme. The first digit is the **major version number,** which is incremented at Torvalds's discretion for each kernel release that contains a feature set significantly different from that of the previous version. Kernels that are described by an even **minor version number** (the digit directly following the first decimal point), such as version 1.0.9, are considered to be stable releases, whereas an odd minor version number, such as 2.1.6, indicates a development version. The digit following the second decimal point is incremented for each minor update to the kernel.

Development kernels include new features that have not been extensively tested, so they are not sufficiently reliable for production use. Throughout the development process, developers create and test new features; then, once a development kernel becomes stable (i.e., the kernel does not contain any known bugs), Torvalds declares it a release kernel.

By the 1996 release of version 2.0, the Linux kernel had grown to over 400,000 lines of code.[1] Thousands of developers had contributed features and bug fixes, and more than 1.5 million users had installed the operating system.[9] Although this release was appealing to the server market, the vast majority of desktop users were

1 Red Hat version 6.2, which included version 2.0 of the Linux kernel, contained approximately 17 million lines of code. By comparison, Microsoft Windows 95 contained approximately 15 million lines of code and Sun Solaris approximately 8 million.[8]

still reluctant to use Linux as a client operating system. Version 2.0 provided enterprise features such as support for SMP, network traffic control and disk quotas. Another important feature allowed portions of the kernel to be modularized, so that users could add device drivers and other system components without rebuilding the kernel.

Version 2.2 of the kernel, which was released by Torvalds in 1999, improved the performance of existing 2.0 features, such as SMP, audio support and file systems, and added new features such as an extension to the kernel's networking subsystem that allowed system administrators to inspect and control network traffic at the packet level. This feature simplified firewall installation and network traffic forwarding, as requested by server administrators.[10]

Many new features in version 2.2, such as USB support, CD-RW support and advanced power management, targeted the desktop market. These features were labeled as experimental, because they were not sufficiently reliable for use in production systems. Although version 2.2 improved usability in desktop environments. Linux could not yet truly compete with popular desktop operating systems of the time, such as Microsoft's Windows 98. The desktop user was more concerned with the availability of applications and the "look and feel" of the user interface than with kernel functionality. However, as Linux kernel development continued, so did the development of Linux applications.

The next stable kernel, version 2.4, was released by Torvalds in January, 2001 In this release a number of kernel subsystems were modified and, in some cases. completely rewritten to support newer hardware and to use existing hardware more efficiently. In addition, Linux was modified to run on high-performance architectures including Intel's 64-bit Itanium, 64-bit MIPS and AMD's 64-bit Opteron, and handheld-device architectures such as SuperH.

Enterprise systems companies such as IBM and Oracle had become increasingly interested in Linux as it continued to stabilize and spread to new platform- Viability in the enterprise systems market, however, required Linux to scale to both high-end and embedded systems, a need fulfilled by version 2.4.[11]

Version 2.4 addressed a critical scalability issue by improving performance on high-end multiprocessor systems. Although Linux had included SMP support since version 2.0, inefficient synchronization mechanisms and other issues limited performance on systems containing more than four processors. Improvements in version 2.4 enabled the kernel to scale to 8,16 or more processors.[12]

Version 2.4 also addressed the needs of desktop users. Experimental feature- in the 2.2 kernel, such as USB support and power management, matured in the 2.4 kernel. This kernel supported a large set of desktop devices; however, a variety of issues, such as Microsoft's market power and the small number of user-friendly Linux applications, prevented widespread Linux use on desktop computers.

Development of the version 2.6 kernel focused on scalability, standards compliance and modifications to kernel subsystems to improve performance. Kernel developers focused on scalability by increasing SMP support, providing support for

NUMA systems and rewriting the process scheduler to increase the performance of scheduling operations. Other kernel enhancements included support for advanced disk scheduling algorithms, a new block I/O layer, improved POSIX compliance, an updated audio subsystem and support for large memories and disks.

20.3 Linux Overview

Linux has a distinct development process and benefits from a wealth of diverse (and free) system and user applications. In this section we summarize Linux kernel features, discuss the process of standardizing and developing the kernel, and introduce several user applications that improve Linux usability and productivity.

In addition to the kernel, Linux systems include user interfaces and applications. A user interface can be as simple as a text-based shell, though standard Linux distributions include a number of GUIs through which users can interact with the system. The Linux operating system borrows from the UNIX layered system approach. Users access applications via a user interface; these applications access resources via a system call interface, thereby invoking the kernel. The kernel may then access the system's hardware, as appropriate, on behalf of the requesting application. In addition to creating user processes, the system creates kernel threads that perform many kernel services. Kernel threads are implemented as **daemons,** which remain dormant until the scheduler or another component of the kernel wakes them.

Because Linux is a multiuser system, the kernel must provide mechanisms to manage user access rights and provide protection for system resources. Therefore, Linux restricts operations that may damage the kernel and/or the system's hardware to a user that has **superuser** (also called **root)** privileges. For example, the superuser privilege enables a user to manage passwords, specify access rights for other users and execute code that modifies system files.

20.3.1 Development and Community

The Linux project is maintained by Linus Torvalds, who is the final arbiter of any code submitted for the kernel. The community of developers constantly modifies the operating system and every two or three years releases a new stable version of the kernel. The community then shifts to the development of the next kernel, discussing new features via e-mail lists and online forums. Torvalds delegates maintenance of stable kernels to trusted developers and manages the development kernel. Bug fixes and performance enhancements for stable releases are applied to the source code and released as updates to the stable version. In parallel, development kernels are released at various stages of the coding process for public review, testing and feedback.

Torvalds and a team of approximately 20 members of his "inner circle"—a set of developers who have proven their competency by producing significant additions to the Linux kernel—are entrusted with enhancing current features and coding new

ones. These primary developers submit code to Torvalds, who reviews and accepts or rejects it, depending on such factors as correctness, performance and style. When a development kernel has matured to a point at which Torvalds is satisfied with the content of its feature set, he will declare a **feature freeze.** Developers may continue to submit bug fixes, code that improves system performance and enhancements to features that are under development.[13] When the kernel is near completion, a **code freeze** occurs. During this phase only code that fixes bugs is accepted. When Torvalds decides that all important known bugs have been addressed, the kernel is declared stable and is released with a new, even kernel minor version number.

Though many Linux developers contribute to the kernel as individuals, corporations such as IBM have invested significant resources in improving the Linux kernel for use in large-scale systems. Such corporations typically charge for tools and support services. Free support is provided by other users and developers in the Linux community. Users may ask questions in user groups, electronic mailing lists (also called listservs) or forums, and may find answers to questions in FAQs (frequently asked questions) and HOWTOs (step-by-step guides). URLs for such resources can be found via the sites in the Web Resources section at the end of the chapter. Alternatively, dedicated support services can be purchased from vendors.

Linux is free for users to download, modify and distribute under the GNU General Public License (GPL). GNU (pronounced *guh-knew)* is a project created by the Free Software Foundation in 1984 that aims to provide free UNIX-like operating systems and software to the public.[14] The General Public License specifies that any distribution of the software under its license must be accompanied by the GPL, must clearly indicate that the original code has been modified and must include the complete source code. Although Linux is free software, it is copyrighted (many of the copyrights are held by Linus Torvalds); any software that borrows from Linux's copyrighted material must clearly credit its source and must also be distributed under the terms of the GPL.

20.3.2 Distributions

By the end of the 1990s, Linux had matured but was still largely ignored by desktop users. In a PC market dominated by Microsoft and Apple, Linux was considered too difficult to use. Those who wished to install Linux were required to download the source code, manually customize configuration files and compile the kernel. Users still needed to download and install applications to perform productive work As Linux matured, developers realized a need for a friendly installation process, which led to the creation of distributions that included the kernel, applications and user interfaces as well as other tools and accessories.

Currently more than 300 distributions are available, each providing a variety of features. User-friendly and application-rich distributions are popular among users — they often include an intuitive GUI and productivity applications such as word processors, spreadsheets and Web browsers. Distributions are commonly divided into **packages,** each containing a single application or service. Users can customize a

Linux system by installing or removing packages, either during the installation process or at runtime. Examples of such distributions are Debian, Mandrake, Red Hat, Slackware and SuSE.[15] Mandrake, Red Hat and SuSE are commercial organizations that provide Linux distributions for markets such as high-end servers and desktop users.[16][17][18] Debian and Slackware are nonprofit organizations comprised of volunteer developers who update and maintain Linux distributions.[19]-[20] Other distributions tailor to specific environments, such as handheld systems (e.g., OpenZaurus) and embedded systems (e.g., uClinux).[21]-[22] All parts of distributions using GPL-icensed code can be freely modified and redistributed by end-users, but the GPL does not prohibit distributors from charging a fee for distribution costs (e.g., the cost if packaging materials) or technical support.[23]-[24]

20.3.3 User Interface

In a Microsoft Windows XP or Macintosh OS X environment, the user is presented with a standard, customizable user interface composed of the GUI and an emulated terminal or shell (e.g., a window containing a command-line prompt). On the contrary, Linux is simply the kernel of an operating system and does not specify a "standard" user interface. Many console shells, such as *bash* (Bourne-again shell), *csh* (a shell providing C-like syntax, pronounced "seashell") and *esh* (easy shell) are commonly found on user systems.[25]

For users who prefer a graphical interface to console shells, there are several freely available GUIs, many of which are packaged as part of most Linux distributions. Those most commonly found in Linux systems are composed of several layers. In most Linux systems, the lowest layer is the X Window System (www.XFree86.org), a low-level graphical interface originally developed at MIT in 1984.[26] The X Window System provides to higher layers the mechanisms necessary to create and manipulate windows and other graphical components. The second layer of the GUI is the **window manager,** which builds on mechanisms in the X Window System interface to control the placement, appearance, size and other attributes of windows. An optional third layer is called the **desktop environment.** The most popular desktop environments are KDE (K Desktop Environment) and GNOME (GNU Network Object Model Environment). Desktop environments tend to provide a file management interface, tools to facilitate access to common applications and utilities, and a suite of software, typically including Web browsers, text editors and e-mail applications.[27]

20.7.4 Standards

A more recent goal of the Linux operating system has been to conform to a variety of widely recognized standards to improve compatibility between applications written for UNIX-like operating systems and Linux. The most prominent set of standards to which Linux developers strive to conform is POSIX (standards.ieee.org/regauth/posi x/).Two other sets prominent in UNIX-like operating systems are the Single UNIX Specification (SUS) and the Linux Standards Base (LSB).

The **Single UNIX Specification** (www.unix.org/version3/) is a suite of standards that define user and application programming interfaces for UNIX operating systems, shells and utilities. Version 3 of the SUS combines several standards (including POSIX, ISO standards and previous versions of the SUS) into one.[28] The Open Group (www.unix.org), which holds the trademark rights and defines standards for the UNIX brand, maintains SUS. To bear the UNIX trademarked name, an operating system must conform to the SUS; The Open Group certifies SUS conformance for a fee.[29]

The **Linux Standard Base** (www.linuxbase.org) is a project that aims to standardize Linux so that applications written for one LSB-compliant distribution will compile and behave exactly the same on any other LSB-compliant distribution The LSB maintains general standards that apply to elements of the operating system, including libraries, package format and installation, commands and utilities. For example, the LSB specifies a standard file system structure. The LSB also maintains architecture-specific standards that are required for LSB certification. Those who wish to test and certify a distribution for LSB compliance can obtain the tools and certification from the LSB organization for a fee.[30]

Until recently, standards compliance has been a low priority for the kernel, because most kernel developers are concerned with improving the feature set and reliability of Linux. Consequently, most kernel releases do not conform to any one set of standards. During the development of the version 2.6 Linux kernel, developers modified several interfaces to improve compliance with the POSIX, SUS and LSB standards.

20.4 Kernel Architecture

Although Linux is a monolithic kernel (see Section 1.13, Operating System Architectures), recent scalability enhancements have included modular capabilities similar to those supported by microkernel operating systems.[31] Linux is commonly referred to as a UNIX-like or a UNIX-based operating system because it provides many services that characterize UNIX systems, such as AT&T's UNIX System V and Berkeley's BSD. Linux is composed of six primary subsystems: process management, interprocess communication, memory management, file system management, I/O management and networking. These six subsystems are responsible for controlling access to system resources (Fig. 20.1). In the following sections, we examine these kernel subsystems and their interactions.

Process execution on a Linux system occurs in either user mode or kernel mode. User processes run in user mode and must therefore access kernel services via the system call interface. When a user process issues a valid system call (in user mode), the kernel executes the system call in kernel mode on behalf of the process. If the request is invalid (e.g., a process attempts to write to a file that is not open). the kernel returns an error.

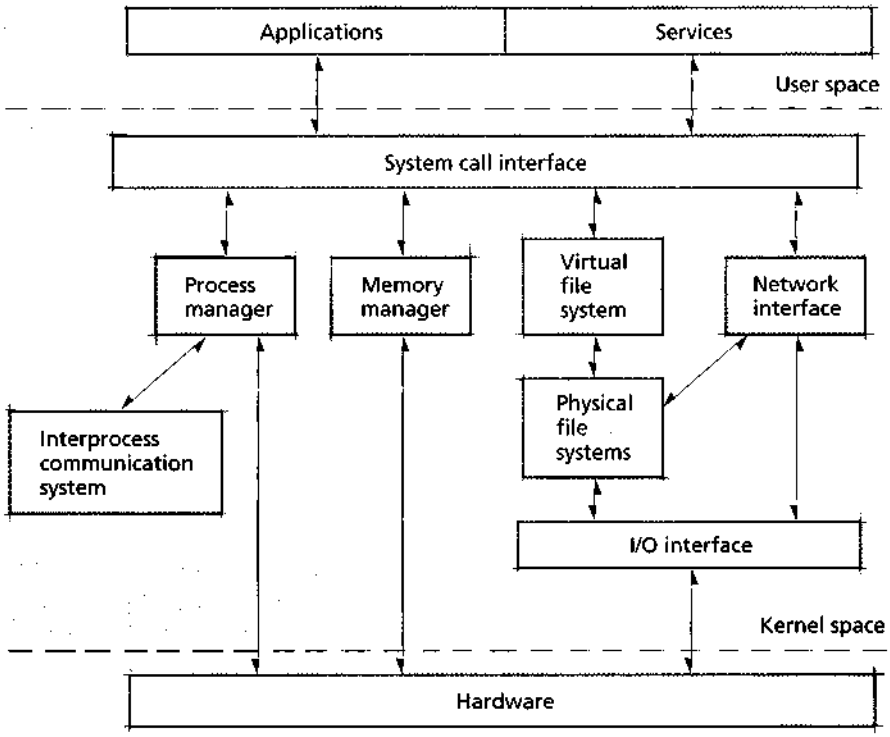

Figure 20.1 | *Linux architecture.*

The process manager is a fundamental Linux subsystem that is responsible for creating processes, providing access to the system's processor(s) and removing processes from the system upon completion (see Section 20.5, Process Management). The kernel's interprocess communication (IPC) subsystem allows processes to communicate with one another. This subsystem interacts with the process manager to permit information sharing and message passing using a variety of mechanisms, discussed in Section 20.10, Interprocess Communication.

The memory management subsystem provides processes with access to memory. Linux assigns each process a virtual memory address space, which is divided into the user address space and the kernel address space. Including the kernel address space within each execution context reduces the cost of context switching from user mode to kernel mode because the kernel can access its data from every user process's virtual address space. The algorithms to manage free (i.e., available) memory and select pages for replacement are discussed in Section 20.6, Memory Management.

Users access files and directories by navigating the directory tree. The root of the directory tree is called the root directory. From the root directory, users can navigate any available file systems. User processes access file system data through

the system call interface. When system calls access a file or directory in the directory tree, they do so through the **virtual file system (VFS)** interface, which provides to processes a single interface to access files and directories stored in multiple heterogeneous file systems (e.g., ext2 and NFS). The virtual file system passes requests to particular file systems, which manage the layout and location of data, as discussed in Section 20.7, File Systems.

Based on the UNIX model, Linux treats most devices as files, meaning that they are accessed using the same mechanisms with which data files are accessed. When user processes read from or write to devices, the kernel passes requests to the virtual file system interface, which then passes requests to the I/O interface. The I/O interface passes requests to device drivers that perform I/O operations on the hardware in a system. In Section 20.8, Input/Output Management, we discuss the I/O interface and its interaction with other kernel subsystems.

Linux provides a networking subsystem to allow processes to exchange data with other networked computers. The networking subsystem accesses the I/O interface to send and receive packets using the system's networking hardware. It allows applications and the kernel to inspect and modify packets as they traverse the system's networking layers via the packet filtering interface. This interface allows systems to implement firewalls, routers and other network utilities. In Section 20.11. Networking, we discuss the various components of the networking subsystem and their implementations.

20.4.1 Hardware Platforms

Initially, Torvalds developed Linux for use on 32-bit Intel x86 platforms. As its popularity grew, developers implemented Linux on a variety of other architectures. The Linux kernel supports the following platforms: x86 (including Intel IA-32), HP Compaq Alpha AXP, Sun SPARC, Sun UltraSPARC, Motorola 68000, PowerPC, PowerPC64, ARM, Hitachi SuperH, IBM S/390 and zSeries, MIPS, HP PA-RISC, Intel IA-64, AMD x86-64, H8/300, V850 and CRIS.[3]2

Each architecture typically requires that the kernel use a different set of low-level instructions to perform operating system functions. For example, an Intel processor implements a different system call mechanism than a Motorola processor. The code that performs operations that are implemented differently across architectures is called **architecture-specific code.** The process of modifying the kernel to support new architecture is called **porting.** To facilitate the process of porting Linux to new platforms, architecture-specific code is separated from the rest of the kernel code into the `/arch` directory of the kernel source tree. The kernel **source tree** organizes each significant component of the kernel into different subdirectories. Each subdirectory in `/arch` contains code corresponding to a particular architecture (e.g., machine instructions for a particular processor). When the kernel must perform processor-specific operations, such as manipulating the contents of a processor cache, control passes to the architecture-specific code that was integrated into the kernel at compile time.[33] Although Linux relies on architecture-specific

code to control computer hardware, Linux may also be executed on a set of virtual hardware devices. The Mini Case Study, User-Mode Linux (UML), describes one such Linux port.

For a system to execute properly on a particular architecture, the kernel must be ported to that architecture and compiled for a particular machine prior to execution. Likewise, applications may need to be compiled (and sometimes redesigned) to properly operate on a particular system. For many platforms, this work has already been accomplished—a variety of platform-specific distributions provide ports of common applications and system services.[34]

Mini Case Study

User-Mode Linux (UML)

Kernel development is a complicated and error-prone process that can result in numerous bugs. Unlike other software, the kernel may execute privileged instructions, meaning that a flaw in the kernel could damage a system's data and hardware. As a result, kernel development can be a tedious (and risky) endeavor. **User-Mode Linux (UML)** facilitates kernel development by allowing developers to test and debug the kernel without damaging the system on which it runs.

User-Mode Linux (UML) is a version of the Linux kernel that runs as a user application on a computer running Linux. Unlike most versions of Linux, which contain architecture-specific code to control devices, UML performs all architecture-specific operations using system calls to the Linux system on which it runs. As a result, UML is interestingly considered to be port of Linux to itself.[35]

The UML kernel runs in user mode, so it cannot execute privileged instructions available to the host kernel. Instead of controlling physical resources, the UML kernel creates virtual devices (represented as files on the host system) that simulate real devices. Because the UML kernel does not control any real hardware, it cannot damage the system.

Almost all kernel mechanisms, such as process scheduling and memory management, are handled in the UML kernel; the host kernel executes only when privileged access to hardware is required. When a UML process issues a system call, the UML kernel intercepts and handles it before it can be sent to the host system. Although this technique incurs significant overhead, UML's primary goal is to provide a safe (i.e., protected) environment in which to execute software, not to provide high performance.[36]

The UML kernel has been applied to more than just testing and debugging. For example, UML can be used to run multiple instances of Linux at once. It can also be used to port Linux such that it runs as an application in operating systems other than Linux. This could allow users to run Linux on top of a UNIX or a Windows system. The Web Resources section at the end of this chapter provides a link to a Web site that documents UML usage and development.

20.4.2 Loadable Kernel Modules

Adding functionality to the Linux kernel, such as support for a particular file system or a new device driver, can be tedious. Because the kernel is monolithic, drivers and file systems are implemented in kernel space. Consequently, to permanently add support for a device driver, users must patch the kernel source by adding the driver code, then recompiling the kernel. This can be a lengthy and error-prone process, so an alternative method has been developed for adding features to the kernel—**loadable kernel modules.**

A kernel module contains object code that, when loaded, is dynamically linked to a running kernel (see Section 2.8, Compiling, Linking and Loading). If a device driver or file system is implemented as a loadable kernel module, it can be loaded into the kernel on demand (i.e., when first accessed) without any additional kernel configuration or compilation. Also, because modules can be loaded on demand, moving code from the kernel into modules reduces the **memory footprint** of the kernel; hardware and file system drivers are not loaded into memory until needed. Modules execute in kernel mode (as opposed to user mode) so they can access kernel functions and data structures. Consequently, loading an improperly coded module can lead to disastrous effects in a system, such as data corruption.[37]

When a module is loaded, the module loader must resolve all references to kernel functions and data structures. Kernel code allows modules to access functions and data structures by exporting their names to a symbol table.[38] Each entry in the symbol table contains the name and address of a kernel function or data structure. The module loader uses the symbol table to resolve references to kernel code.[39]

Because modules execute in kernel mode, they require access to symbols in the kernel symbol table, which allows modules to access kernel functions. However, consider what can happen if a function is modified between kernel versions. If a module was written for a prior kernel version, the module may expect a particular, yet invalid, result from a current kernel function (e.g., an integer value instead of an unsigned long value), which may in turn lead to errors such as exceptions. To avoid this problem, the kernel prevents users from loading modules written for a version of the kernel other than the current one, unless explicitly overridden by the superuser.[40]

Modules must be loaded into the kernel before use. For convenience, the kernel supports dynamic module loading. When compiling the kernel, the user is given the option to enable or disable *kmod—a* kernel subsystem that manages modules without user intervention. The first time the kernel requires access to a module, it issues a request to *kmod* to load the module. *Kmod* determines any module dependencies, then loads the requested module. If a requested module depends on other modules that have not been loaded, *kmod* will load those modules on demand.[41]

20.5 Process Management

The process management subsystem is essential to providing efficient multiprogramming in Linux. Although responsible primarily for allocating processors to processes,

the process management subsystem also delivers signals, loads kernel modules and receives interrupts. The process management subsystem contains the process scheduler, which provides processes access to a processor in a reasonable amount of time.

20.5.1 Process and Thread Organization

In Linux systems, both processes and threads are called tasks; internally, they are represented by a single data structure. In this section, we distinguish processes from threads from tasks where appropriate. The process manager maintains a list of all tasks using two data structures. The first is a circular, doubly linked list in which each entry contains pointers to the previous and next tasks in the list. This structure is accessed when the kernel must examine all tasks in the system. The second is a hash table. When a task is created, it is assigned a unique **PID** (process identifier). Process identifiers are passed to a hash function to determine their location in the process table. The hashing method provides quick access to a specific task's data structure when the kernel knows its PID.[42]

Each task in the process table is represented by a `task_struct` structure, which serves as the process descriptor (i.e., the PCB). The `task_struct` structure stores variables and nested structures containing information describing a process. For example, the variable `state` stores information about the current task state. *[Note:* The kernel is primarily written using the C programming language and makes extensive use of structures to represent software entities.]

A task transitions to the *running* state when it is dispatched to a processor (Fig. 20.2). A task enters the *sleeping* state when it blocks and the *stopped* state when it is suspended. The *zombie* state indicates that a task has been terminated but has not yet been removed from the system. For example, if a process contains several threads, it will enter the *zombie* state until its threads have been notified that it received a termination signal. A task in the *dead* state may be removed from the system. The states *active* and *expired* are process scheduling states (described in the next section), which are not stored in the variable `state`.

Other important task-specific variables permit the scheduler to determine when a task should run on a processor. These variables include the task's priority, whether the task is a real-time task and, if so, which real-time scheduling algorithm should be used (real-time scheduling is discussed in the next section).[43]

Nested structures within a `task_struct` store additional information about a task. One such structure, mm_struct, describes the memory allocated to a task (e.g., the location of its page table in memory and the number of tasks sharing its address space). Additional structures nested within a `task_struct` contain information such as register values that store a task's execution context, signal handlers and the task's access rights.[44] These structures are accessed by several kernel subsystems other than the process manager.

When the kernel is booted, it typically loads a process called *init,* which then uses the kernel to create all other tasks.[45] Tasks are created using the `clone` system call; any calls to `fork` or `vfork` are converted to clone system calls at compile

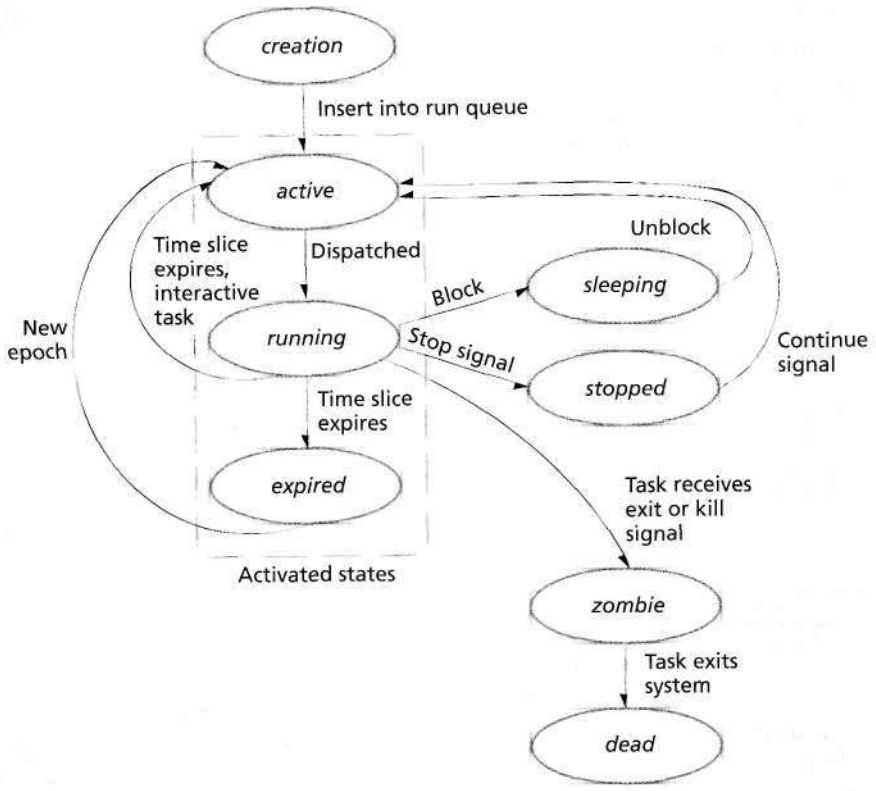

Figure 20.2 | Task state-transition diagram.

time.The purpose of fork is to create a child task whose virtual memory space is allocated using copy-on-write to improve performance (see Section 10.4.6, Sharing in a Paging System). When the child or the parent attempts to write to a page in memory, the writer is allocated its own copy of the page. As discussed in Section 10.4.6, copy-on-write can lead to poor performance if a process calls execve to load a new program immediately after the fork. For example, if the parent executes before its child, a copy-on-write will be performed for any page the parent modifies. Because the child will not use any of its parent's pages (if the child will immediately call execve when it executes), this operation is pure overhead. Therefore, Linux supports the vfork call, which improves performance when child processes will call execve. vfork suspends the parent process until the child calls execve or exi t, to ensure that the child loads its new pages before the parent causes any wasteful copy-on-write operations, vfork further improves performance by not copying the parent's page tables to the child, because new page table entries will be created when the child calls execve.

*Linux Threads and the **clone** System Call*

Linux provides support for threads using the clone system call, which enables the calling process to specify whether the thread shares the process's virtual memory, file system information, file descriptors and/or signal handlers.[46] In Section 4.2, Definition of Thread, we mentioned that processor registers, the stack and other thread-specific data (TSD) are local to each thread, while the address space and open file handles are global to the process that contains the threads. Thus, depending on how many of the process's resources are shared with its thread, the resulting thread may be quite different from the threads described in Chapter 4.

Linux's implementation of threads has generated much discussion regarding the definition of a thread. Although clone creates threads, they do not precisely conform to the POSIX thread specification (see Section 4.8, POSIX and Pthreads). For example, two or more threads that were created using a **clone** call specifying maximum resource sharing still maintain several data structures that are not shared with all threads in the process, such as access rights.[47]

When clone is called from a kernel process (i.e., a process that executes kernel code), it creates a **kernel thread** that differs from other threads in that it directly accesses the kernel's address space. Several daemons within the kernel are implemented as kernel threads—these daemons are services that sleep until awakened by the kernel to perform tasks such as flushing pages to secondary storage and scheduling software interrupts (see Section 20.8.6, Interrupts).[48] These tasks are generally maintenance related and execute periodically.

There are several benefits to the Linux thread implementation. For example, Linux threads simplify kernel code and reduce overhead by requiring only a single copy of task management data structures.[49] Moreover, although Linux threads are less portable than POSIX threads, they allow programmers the flexibility to tightly control shared resources between tasks. A recent Linux project, Native POSIX Thread Library (NPTL), has achieved nearly complete POSIX conformance and is likely to become the default threading library in future Linux distributions.[50]

20.5.2 Process Scheduling

The goal of the Linux process scheduler is to run all tasks within a reasonable amount of time while respecting task priorities, maintaining high resource utilization and throughput, and reducing the overhead of scheduling operations. The process scheduler also addresses Linux's role in the high-end computer system market by scaling to SMP and NUMA architectures while providing high processor affinity. One of the more significant scalability enhancements in version 2.6 is that all scheduling functions are constant-time operations, meaning that the time required to execute scheduling functions does not depend on the number of tasks in the system.[51]

At each system timer interrupt (an architecture-specific number set to 1 millisecond by default for the IA-32 architecture[52]), the kernel updates various bookkeeping data structures (e.g., the amount of time a task has been executing) and performs scheduling operations as necessary. Because the scheduler is preemptive,

each task runs until its quantum, or time slice, expires, a higher priority process becomes runnable or the process blocks. Each task's time slice is calculated as a function of the process's priority upon release of the processor (with the exception of real-time tasks, which are discussed later). To prevent time slices from being too small to allow productive work or so large as to diminish response times, the scheduler ensures that the time slice assigned to each task is between 10 and 200 timer intervals, corresponding to a range of 10 to 200 milliseconds on most systems (like most scheduler parameters, these default values have been chosen empirically). When a task is preempted, the scheduler saves the task state to its `task_struct` structure. If the process's time slice has expired, the scheduler recalculates the process's priority, determines the task's next time slice and dispatches the next process.

Run Queues

Once a task has been created using `clone`, it is placed in a processor's **run queue.** which contains references to all tasks competing for execution on that processor. Run queues, similar to multilevel feedback queues (Section 8.7.6, Multilevel Feedback Queues), assign tasks to priority levels. The **priority array** maintains pointers to each level of the run queue. Each entry in the priority array points to a list of tasks—a task of priority *i* is placed in the ith entry of a priority array in the run queue (Fig. 20.3).

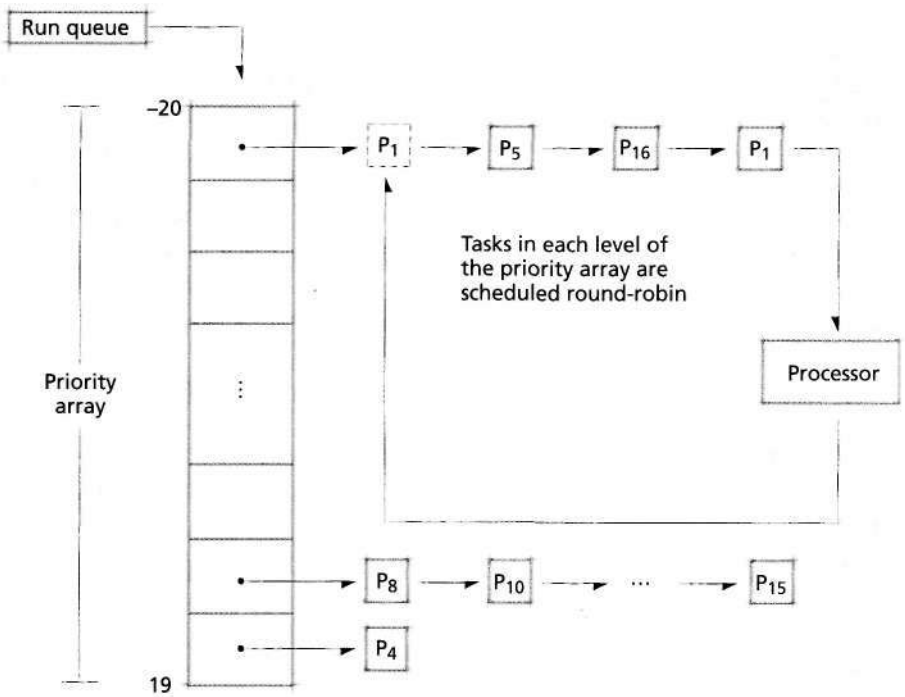

Figure 20.3 | *Scheduler priority array.*

The scheduler dispatches the task at the front of the list in the highest level of the priority array. If more than one task exists in a level of the priority array, tasks are dispatched from the priority array round-robin. When a task enters the *blocked* or *sleeping* (i.e., *waiting)* state, or is otherwise unable to execute, that task is removed from its run queue.

One goal of the scheduler is to prevent indefinite postponement by defining a period of time called an **epoch** during which each task in the run queue will execute at least once. To distinguish processes that are considered for processor time from those that must wait until the next epoch, the scheduler defines an ***active* state** and an ***expired* state.** The scheduler dispatches only processes in the *active* state.

The duration of an epoch is determined by the **starvation limit—**an empirically derived value that provides high-priority tasks with good response times while ensuring that low-priority tasks are dispatched often enough to perform productive work within a reasonable amount of time. By default, the starvation limit is set to lOn seconds, where *n* is the number of tasks in the run queue. When the current epoch has lasted longer than the starvation limit, the scheduler transitions each active task in the run queue to the *expired* state (the transition occurs after each active task's time slice expires). This temporarily suspends high-priority tasks (with the exception of real-time tasks), allowing low-priority tasks to execute. When all tasks in the run queue have executed at least once, all tasks in that run queue will be in the *expired* state. At this point, the scheduler transitions all tasks in the run queue to the *active* state and a new epoch begins.[53]

To simplify the transition from the *expired* state to the *active* state at the end of an epoch, the Linux scheduler maintains two priority arrays for each processor. The priority array that contains tasks in the active state is called the **active list.** The priority array that stores expired tasks (i.e., tasks that are not allowed to execute until the next epoch) is called the **inactive** (or **expired) list.** When a task transitions from the *active* state to the *expired* state, it is placed in the level of the expired list's priority array corresponding to its priority when it transitioned to the *expired* state. At the end of an epoch, all tasks are located in the *expired* state and must transition to the *active* state. The scheduler performs this operation quickly by simply swapping the pointers to the expired list and the active list. By maintaining two priority arrays per process, the scheduler can transition all tasks in a run queue using a single swap operation, a performance enhancement that generally outweighs the nominal memory overhead due.[54]

The Linux scheduler scales to multiprocessor systems by maintaining one run queue for each physical processor in the system. One reason for per-processor run queues is to assign tasks to execute on particular processors to exploit processor affinity. Recall from Chapter 15 that processes in some multiprocessor architectures, such as NUMA, achieve higher performance when a task's data is stored in a processor's local memory and in a processor's cache. Consequently, tasks can achieve higher performance if they are consistently assigned to a single processor (or node). However, per-processor run queues risk unbalancing processor loads, leading to

reduced system performance and throughput (see Section 15.7, Process Migration). Later in this section, we discuss how Linux addresses this issue by dynamically balancing the number of tasks executing on each processor in the system.

Scheduling Priority

In the Linux scheduler, a task's priority affects the size of its time slice and the order in which it executes on a processor. Upon creation, tasks are assigned a **static priority,** also called the **nice value.** The scheduler recognizes 40 distinct priority levels, ranging from -20 to 19. Conforming to UNIX convention, smaller priority values denote higher priority in the scheduling algorithm (i.e., -20 is the highest priority a process can attain).

One goal of the Linux scheduler is to provide a high level of system interactivity. Because interactive tasks typically block to perform I/O or sleep (e.g., while waiting for a user response), the scheduler dynamically boosts the priority (by decrementing the static priority value) of a task that yields its processor before the task's time slice expires. This is acceptable because I/O-bound processes normally use the processor only briefly before generating an I/O request. Thus, giving I/O-bound tasks high priority has little effect on processor-bound tasks, which might use the processor for hours at a time if the system makes the processor available on a nonpreemptible basis. The modified priority level is called a task's **effective priority.** which is calculated when a task sleeps or consumes its time slice. A task's effective priority determines the level of the priority array in which a task is placed. Therefore, a task that receives a priority boost is placed in a lower level of the priority array, meaning it will execute before tasks of a higher effective priority value.

To further improve interactivity, the scheduler penalizes a processor-bound task by increasing its static priority value. This places a processor-bound task in a higher level of the priority array, meaning tasks of a smaller effective priority will be executed before it. Again, this ultimately has little effect on processor-bound tasks because the higher-priority interactive tasks execute only briefly before blocking.

To ensure that a task executes at or near the priority it was initially assigned. the task scheduler does not allow a task's effective priority to differ from its static priority by more than five units. In this sense, the scheduler honors the priority levels assigned to a task when it was created.

Scheduling Operations

The scheduler removes a task from a processor if the task is interrupted, preempted (e.g., if its time slice expires) or blocks. Each time a task is removed from a processor, the scheduler calculates a new time slice. If the task blocks, or is otherwise unable to execute, it is **deactivated,** meaning that it is removed from the run queue until it becomes ready to execute. Otherwise, the scheduler determines whether the task should be placed in the active list or the inactive list. The algorithm that determines this has been empirically derived to provide good performance; its primary factors are a task's static and effective priorities.

The result of the algorithm is depicted in Fig. 20.4. The y-axis of Fig. 20.4 indicates a task's static priority value and the x-axis represents a task's priority adjustment (i.e., boost or penalty) The shaded region indicates sets of static priority values and priority adjustments that cause a task to be rescheduled, meaning that it is placed at the end of its corresponding priority array in the active list. In general, if a task is of high priority and/or has received a significant bonus to its effective priority, it is rescheduled. This allows high-priority, I/O-bound and interactive tasks to execute more than once per epoch. In the unshaded region, tasks that are of low priority and/or have received priority penalties are placed in the expired list.

When a user process cl ones, it may seem reasonable to allocate each child its own time slice. However, if a task spawns a large number of new tasks, and all of its children are allocated their own time slices, other tasks in the system might experience unreasonably poor response times during that epoch. To improve fairness, Linux requires that each parent process initially share its time slice with its children when a user process clones.The scheduler enforces this requirement by assigning half of the parent's original time slice to both the parent process and its child the child is spawned. To prevent legitimate processes from suffering low levels of ser-

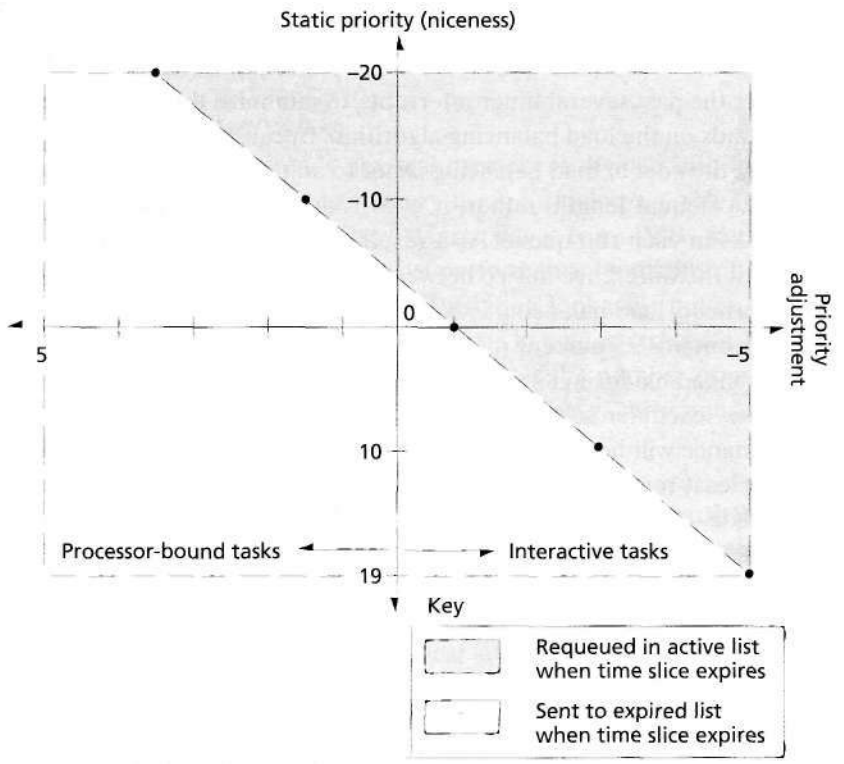

Figure 20.4 | *Priority values and their corresponding levels of interactivity.*[55]

vice due to spawning child processes, this reduced time slice applies only during the remainder of the epoch during which the child is spawned.

Multiprocessor Scheduling

Because the process scheduler maintains tasks in a per-processor run queue, tasks will generally exhibit high processor affinity. This means that a task will likely be dispatched to the same processor for each of its time slices, which can increase performance when a task's data and instructions are located in a processor's caches. However, such a scheme could allow one or several processors on an SMP system to lie idle even during a heavy system load. To avoid this, if the scheduler detects that a processor is idle, it performs **load balancing** to migrate tasks from one processor to another to improve resource utilization. If the system contains only one processor, load balancing routines are removed from the kernel when it is compiled.

The scheduler determines if it should perform load balancing routines after each timer interrupt, which is set to one millisecond on IA-32 systems. If the processor that issued the timer interrupt is idle (i.e., its run queue is empty), the scheduler attempts to migrate tasks from the processor with the heaviest load (i.e., the processor that contains the largest number of processes in its run queue) to the idle processor. To reduce load balancing overhead, if the processor that triggered the interrupt is not idle, the scheduler will attempt to move tasks to that processor every 200 timer interrupts instead of after every timer interrupt.[56]

The scheduler determines processor load by using the average length of each run queue over the past several timer interrupts, to minimize the effect of variations in processor loads on the load balancing algorithm. Because processor loads tend to change rapidly, the goal of load balancing is not to adjust the size of two run queues until they are of equal length; rather, it is to reduce the imbalance between the number of tasks in each run queue. As a result, tasks are removed from the larger run queue until the difference in size between the two run queues has been halved. To reduce overhead, load balancing is not performed unless the run queue with the heaviest load contains 25 percent more tasks than the run queue of the processor performing the load balancing.[57]

When the scheduler selects tasks for balancing, it attempts to choose tasks whose performance will be least affected by moving from one processor to another. In general, the least-recently active task on a processor will most likely be cache-cold on the processor—a **cache-cold** task does not contain much (or any) of the task's data in its processor's cache, whereas a **cache-hot** task contains most (or all) of the task's data in the processor cache. Therefore, the scheduler chooses to migrate tasks that are most likely cache-cold (i.e., tasks that have not executed recently).

Real-Time Scheduling

The scheduler supports soft real-time scheduling by attempting to minimize the time during which a real-time task waits to be dispatched to a processor. Unlike a normal task, which is eventually placed in the expired list to prevent low-priority tasks from being indefinitely postponed, a real-time task is always placed in the

active list after its quantum expires. Further, real-time tasks always execute with higher priority than normal tasks. Because the scheduler always dispatches a task from the highest-priority queue in the active list (and real-time tasks are never removed from the active list), normal tasks cannot preempt real-time tasks.

The scheduler complies with the POSIX specification for real-time processes by allowing real-time tasks to be scheduled using the default scheduling algorithm described in the previous sections, or using the round-robin or FIFO scheduling algorithms. If a task specifies round-robin scheduling and its time slice has expired, the task is allocated a new time slice and is enqueued at the end of its priority array in the active list. If the task specifies FIFO scheduling, it is not assigned a time slice and therefore will execute on a processor until it exits, sleeps, blocks or is interrupted.[58] Clearly, real-time processes can indefinitely postpone other processes if coded improperly, resulting in poor response times. To prevent accidental or malicious misuse of real-time tasks, only users with root privileges can create them.

20.6 Memory Management

During the development of kernel versions 2.4 and 2.6, the memory manager was heavily modified to improve performance and scalability. The memory manager supports both 32- and 64-bit addresses as well as nonuniform memory access (NUMA) architectures to allow it to scale from desktop computers and workstations to servers and supercomputers.

20.6.1 Memory Organization

On most architectures, a system's physical memory is divided into fixed-size page frames. Generally, Linux allocates memory using a single page size (often 4KB or 8KB); on some architectures that support large pages (e.g., 4MB), kernel code may be placed in large pages. This can improve performance by minimizing the number of entries for kernel page frames in the translation lookaside buffer (TLB).[59] The kernel stores information about each page frame in a page structure. This structure contains variables that describe page usage, such as the number of processes sharing the page and flags indicating the state of the page (e.g., dirty, unused, etc).[60]

Virtual Memory Organization

On 32-bit systems, each process can address 2^{32} bytes, meaning that each virtual address space is 4GB. The kernel supports larger virtual address spaces on 64-bit systems—up to 2 petabytes (i.e., 2 million gigabytes) on Intel Itanium processors (the Itanium processor uses only 51 bits to address main memory, but the IA-64 architecture can support up to 64-bit physical addresses).[61] In this section, we focus on the 32-bit implementation of the virtual memory manager. Entries describing the virtual-to-physical address mappings are located in each process's page tables. The virtual memory system supports up to three levels of page tables to locate the mappings between virtual pages and page frames (see Fig. 20.5). The first level of the page table hierarchy, called the **page global directory,** stores addresses of sec-

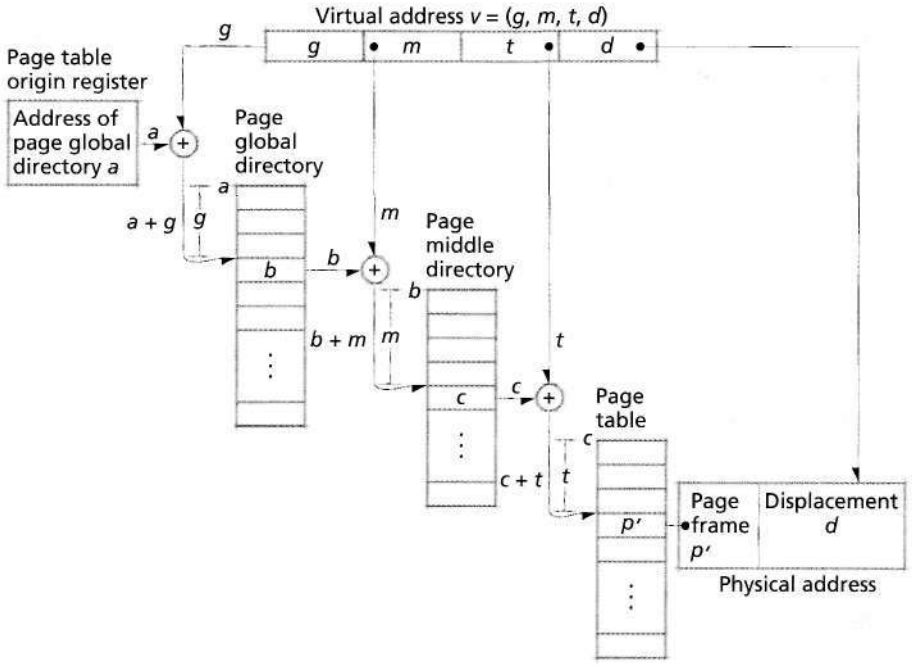

Figure 20.5 | Page table organization.

ond-level tables. Second-level tables, called **page middle directories,** store addresses of the third-level tables. The third level, simply called page tables, maps virtual pages to page frames.

The kernel partitions a virtual address into four fields to provide processors with multilevel page address translation information. The first three fields are indices into the process's page global directory, page middle directory and page table, respectively. These three fields allow the system to locate the page frame corresponding to a virtual page. The fourth field contains the displacement (also called offset) from the physical address of the beginning of the page frame.[62] Linux disables page middle directories when running on the IA-32 architecture, which supports only two levels of page tables when the Physical Address Extension (PAE) feature is disabled. Page middle directories are enabled for 64-bit architectures that support three or more levels of page tables (e.g., the x86-64 architecture, which supports four levels of page tables).

Virtual Memory Areas

Although a process's virtual address space is composed of individual pages, the kernel uses a higher-level mechanism, called **virtual memory areas,** to organize the virtual memory a process is using. A virtual memory area describes a contiguous set of pages in a process's virtual address space that are assigned the same protection

(e.g., read-only, read/write, executable) and backing store. The kernel stores a process's executable code, heap, stack and each memory-mapped file (see Section 13.9, Data Access Techniques) in separate virtual memory areas.[63]-[64]

When a process requests additional memory, the kernel attempts to satisfy that request by enlarging an existing virtual memory area. The virtual memory area the kernel selects depends on the type of memory the process is requesting (e.g., executable code, heap, stack, etc.). If the process requests memory that does not correspond to an existing virtual memory area, or if the kernel cannot allocate a contiguous address space of the requested size in an existing virtual memory area, the kernel creates a new virtual memory area.[65]

Virtual Memory Organization for the IA-32 Architecture

In Linux, virtual memory organization is architecture specific. In this section, we discuss how the kernel organizes virtual memory by default to optimize performance on the IA-32 architecture.

When the kernel performs a context switch, it must provide the processor with page address translation information for the process that is about to execute (see Section 10.4.1). Recall from Section 10.4.3 that an associative memory called the translation lookaside buffer (TLB) stores recently used page table entries (PTEs) so that the processor can quickly perform virtual-to-physical address translations for the process that is currently running. Because each process is allocated a different virtual address space, PTEs for one process are not valid for another. As a result, the PTEs in the TLB must be removed after a context switch. This is called flushing the TLB—the processor removes each PTE from the TLB and updates the PTEs in main memory to match any modified PTEs in the TLB. In particular, each time the value of the page table origin register changes, the TLB must be flushed. The overhead due to a TLB flush can be substantial because the processor must access main memory to update each PTE that is flushed. If the kernel changes the value of the page table origin register to execute each system call, the overhead due to TLB flushing can significantly reduce performance.

To reduce the number of expensive TLB flush operations, the kernel ensures that it can use any process's page table origin register to access the kernel's virtual address space. The kernel does this by dividing each process's virtual address space into user addresses and kernel addresses. The kernel allows each process to access up to 3GB of the process's virtual address space—the virtual addresses from zero to 3GB. Therefore, virtual-to-physical address translation information can vary between processes for the first 3GB of each 32-bit virtual address space. The kernel address space is the remaining 1GB of virtual addresses in each process's 32-bit virtual address space (addresses ranging from 3GB to 4GB), as shown in Fig. 20.6. The virtual-to-physical address translation information for this region of memory addresses does not vary from one process to another. Therefore, when a user process invokes the kernel, the processor does not need to flush the TLB, which improves performance by reducing the number of times the processor accesses main memory to update page table entries.[66]

Often, the kernel must access main memory on behalf of user processes (e.g., to perform I/O); therefore, it must be able to access every page frame in main memory. However, today's processors require that all memory references use virtual addresses to access memory (if virtual memory is enabled). As a result, the kernel generally must use virtual addresses to access page frames.

When using virtual addresses, the kernel must provide the processor with PTEs that map the kernel's virtual pages to the page frames it must access. The kernel could create these PTEs each time it accessed main memory; however, doing so would create significant overhead. Therefore, the kernel creates PTEs that map most of the pages in the kernel's virtual address space permanently to page frames in main memory. For example, the first page of the kernel's virtual address space always points to the first page frame in main memory; the 100th page of the kernel's virtual address space always points to the 100th page frame in main memory (Fig. 20.6).

Note that creating a PTE (i.e., mapping virtual page to a page frame) does not allocate a page frame to the kernel or a user process. For example, assume that page frame number 100 stores a process's virtual page *p*. When the kernel accesses virtual page number 100 in the kernel virtual address space, the processor maps the virtual page number to page frame number 100, which stores the contents of *p*. Thus, the kernel's virtual address space is used to access page frames that may be allocated to the kernel or user processes.

Ideally, the kernel would be able to create PTEs that permanently map to each page frame in memory. However, if a system contains more than 1GB of main memory, the kernel cannot create a permanent mapping to every page frame because it reserves only 1GB of each 4GB virtual address space for itself. For example, when the kernel performs I/O on behalf of a user process, it must be able to access the data using pages in its 1GB virtual address space. However, if a user pro-

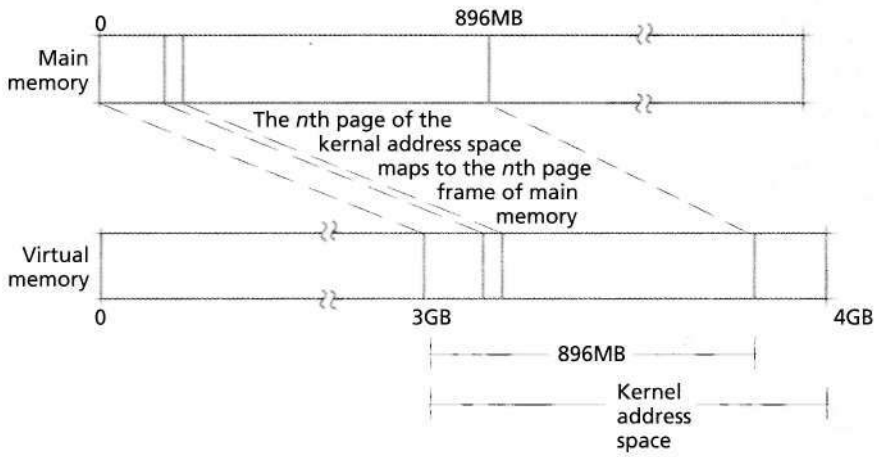

Figure 20.6 | *Kernel virtual address space mapping.*

cess requests I/O for a page that is stored at an address higher than 1GB, the kernel might not contain a mapping to that page. In this case, the kernel must be able to create a temporary mapping between a kernel virtual page and a user's physical page in main memory to perform the I/O. To address this problem, the kernel maps most of its virtual pages permanently to page frames and reserves several virtual pages to provide temporary mappings to the remaining page frames. In particular, the kernel creates PTEs that map the first 896MB of its virtual pages permanently to the first 896MB of main memory when the kernel boots. It reserves the remaining 128MB of its virtual address space for temporary buffers and caches that can be mapped to other regions of main memory. Therefore, if the kernel must access page frames beyond 896MB, it uses virtual addresses in this region to create a new PTE that temporarily maps a virtual page to a page frame.

Physical Memory Organization

The memory management system divides a system's physical address space into three **zones** (Fig. 20.7). The size of each zone is architecture dependent; in this section we present the configuration for the IA-32 architecture discussed in the previous section. The first zone, called **DMA memory,** includes the main memory locations from 0-16MB. The primary reason for creating a DMA memory zone is to ensure compatibility with legacy architectures. For example, some direct memory

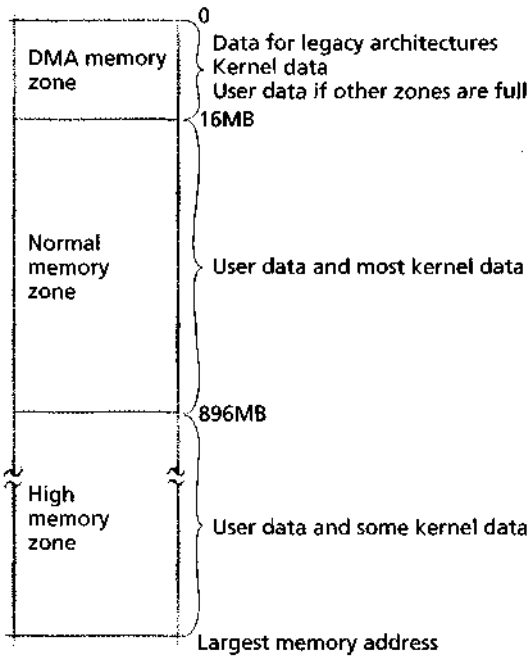

Figure 20.7 | *Physical memory zones on the IA-32 Intel architecture.*

access (DMA) devices can address only up to 16MB of memory, so Linux reserves memory in this zone for such devices. The DMA memory zone also contains kernel data and instructions (e.g., bootstrapping code) and might be allocated for user processes if free memory is scarce.

The second zone, called **normal memory,** includes the physical memory locations between 16MB and up to 896MB. The normal memory zone can be used to store user and kernel pages as well as data from devices that can access memory greater than 16MB using DMA. Note that because the kernel's virtual address space is mapped directly to the first 896MB of main memory, most kernel data structures are located in the low memory zones (i.e., the DMA or normal memory zones). If these data structures were not located in these memory zones, the kernel could not provide a permanent virtual-to-physical address mapping for kernel data and might cause a page fault while executing its code. Page faults not only reduce kernel performance but can be fatal when performing error-handling routines.

The third zone, called **high memory,** includes physical memory locations from 896MB to a maximum of 64GB on Pentium processors. (Intel's Page Address Extension feature enables 36-bit memory addresses, allowing the system to access 2^{36} bytes. or 64GB, of main memory.) High memory is allocated to user processes, any devices that can access memory in this zone and temporary kernel data structures.[67-68]

Some devices, however, cannot access data in high memory because the number of physical addresses they can address is limited. In this case, the kernel copies such data to a buffer, called a **bounce buffer,** in DMA memory to perform I/O. After completing an I/O operation, the kernel copies any modified pages in the buffer to the page in high memory[69-70-71]

Depending on the architecture, the first megabyte of main memory might contain data loaded into memory by initialization functions in the BIOS (see Section 2.3.1, Mainboards). To avoid overwriting such data, the Linux kernel code and data structures are loaded into a contiguous area of physical memory, typically beginning at the second megabyte of main memory. (The kernel reclaims most of the first megabyte of memory.after loading.) Kernel pages are never swapped (i.e.. paged) or relocated in physical memory. In addition to improving performance, the contiguous and static nature of kernel memory simplifies coding for kernel developers at a relatively low cost (the kernel footprint is approximately 2MB).[72]

20.6.2 Physical Memory Allocation and Deallocation

The kernel allocates page frames to processes using the **zone allocator.** The zone allocator attempts to allocate pages from the physical zone corresponding to each request. Recall that the kernel reserves as much of the DMA memory zone as possible for use by legacy architectures and kernel code. Also, performing I/O operations on high memory might require use of a bounce buffer, which is less efficient than using pages that are directly accessible by DMA hardware. Thus, although pages for user processes can be allocated from any zone, the kernel attempts to allocate them first from the high memory zone. If the high memory zone is full and

pages in normal memory are available, then the zone allocator uses pages from normal memory. Only when free memory is scarce in both the normal and the high zone of memory does the zone allocator select pages in DMA memory.[73]

When deciding which page frames to allocate, the zone allocator searches for empty pages in each zone's free_area vector. The free_area vector contains references to a zone's free lists and bitmaps that identify contiguous blocks of memory. Blocks of page frames are allocated in groups of powers of two; each element in a zone's free_area vector contains a list of blocks that are the same size-the nth element in the vector references a list of blocks of size 2^n.[74] Figure 20.8 illustrates the first three entries of the free_area vector.

To locate blocks of the requested size, the memory manager uses the **binary buddy algorithm** to search the f ree_area vector. The buddy algorithm, described by Knowlton and Knuth, is a simple physical page allocation algorithm that provides good performance.[75, 76] If there are no blocks of the requested size, a block of the next-closest size in the free_area vector is halved repeatedly until the resulting block is of the correct size. When the memory manager finds a block of the correct size, it allocates it to the process that requested it and places any orphaned free blocks in the appropriate lists.[77]

When memory is deallocated, the buddy algorithm groups contiguous free pages as follows. When a process frees a block of memory, the memory manager checks the bitmap (in the free_area vector) that tracks blocks of that size. If the bitmap indicates that an adjacent block is free, the memory manager combines the two blocks (buddies) into a larger block. The memory manager repeats this process until there are no blocks with which to combine the resulting block. The memory manager then inserts the block into the proper list in free_area.[78]

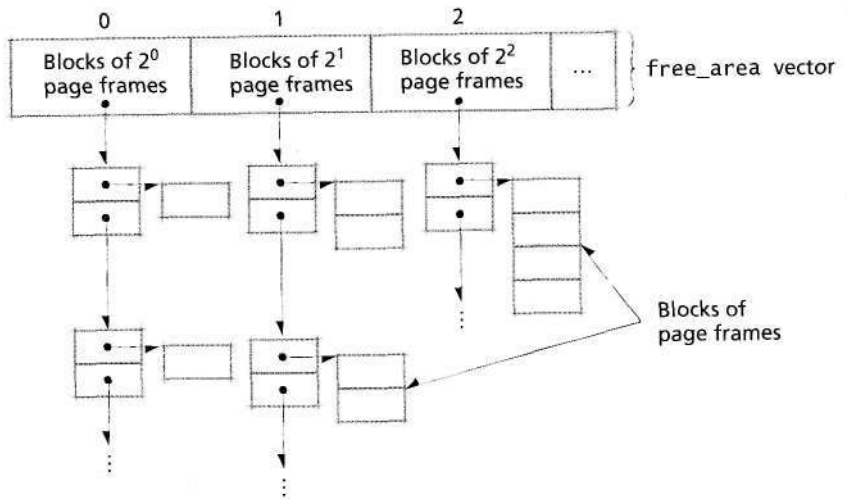

Figure 20.8 | free_area vector.

There are several kernel data structures (e.g., the structure that describes virtual memory areas) that consume much less than a page of memory (4KB is the smallest page size on most systems). Processes tend to allocate and release many such data structures during the course of execution. Requests to the zone allocator to allocate such small amounts of memory would result in substantial internal fragmentation because the smallest unit of memory the zone allocator can supply is a page. Instead, the kernel satisfies such requests via the **slab allocator.** The slab allocator allocates memory from any one of a number of available **slab caches.**[79] A slab cache is composed of a number of objects, called slabs, that span one or more pages and contain structures of the same type. Typically, a **slab** is one page of memory that serves as a container for multiple data structures smaller than a page. When the kernel requests memory for a new structure, the slab allocator returns a portion of a slab in the slab cache for that structure. If all of the slabs in a cache are occupied, the slab allocator increases the size of the cache to include more slabs. These new slabs contain pages allocated using the appropriate zone allocator.[80]

As previously discussed, serious or fatal system errors can occur if the kernel causes a page fault during interrupt- or error-handling code. Similarly, a request to allocate memory while executing such code must not fail if the system contains few free pages. To prevent such a situation, Linux allows kernel threads and device drivers to allocate memory pools. A **memory pool** is a region of memory that the kernel guarantees will be available to a kernel thread or device driver regardless of how much memory is currently occupied. Clearly, extensive use of memory pools limits the number of page frames available to user processes. However, because a system failure could result from a failed memory allocation, the trade-off is justified.[81]

20.6.3 Page Replacement

The Linux memory manager determines which pages to keep in memory and which pages to replace (known as "swapping" in Linux) when free memory becomes scarce. Recall that only pages in the user region of a virtual address space can be replaced; most pages containing kernel code and data cannot be replaced.

As pages are read into memory, the kernel inserts them into the **page cache.** The page cache is designed to reduce the time spent performing disk I/O operations. When the kernel must flush (i.e., write) a page to disk, it does so through the page cache. To improve performance, the page cache employs write-back caching (see Section 12.8, Caching and Buffering) to clean dirty pages.[82]

Each page in the page cache must be associated with a secondary storage device (e.g., a disk) so the kernel knows where to place pages when they are swapped out. Pages that are mapped to files are associated with a file's inode, which describes the file's location on disk (see Section 20.7.1, Virtual File System, for a detailed description of inodes). As we discuss in the next section, pages that are not mapped to files are placed on secondary storage in a region called the system swap file.[83]

When physical memory is full and a nonresident page is requested by processes or the kernel, a page frame must be freed to fill the request. The memory

manager provides a simple, efficient page-replacement strategy. Figure 20.9 illustrates this strategy, fn each memory zone, pages are divided into two groups: **active pages** and **inactive pages.** To be considered active, a page must have been referenced recently. One goal of the memory manager is to maintain the current working set inside the collection of active pages.[84]

Linux uses a variation of the clock page-replacement strategy (see Section 11.6.7). The memory manager uses two linked lists per zone to implement page replacement—the active list contains active pages, the inactive list contains inactive pages. The lists are organized such that the most-recently used pages are near the head of the active list, and the least-recently used pages are near the tail of the inactive list.[85]

When the memory manager allocates a page of memory to a process, the page's associated page structure is placed at the head of that zone's inactive list and

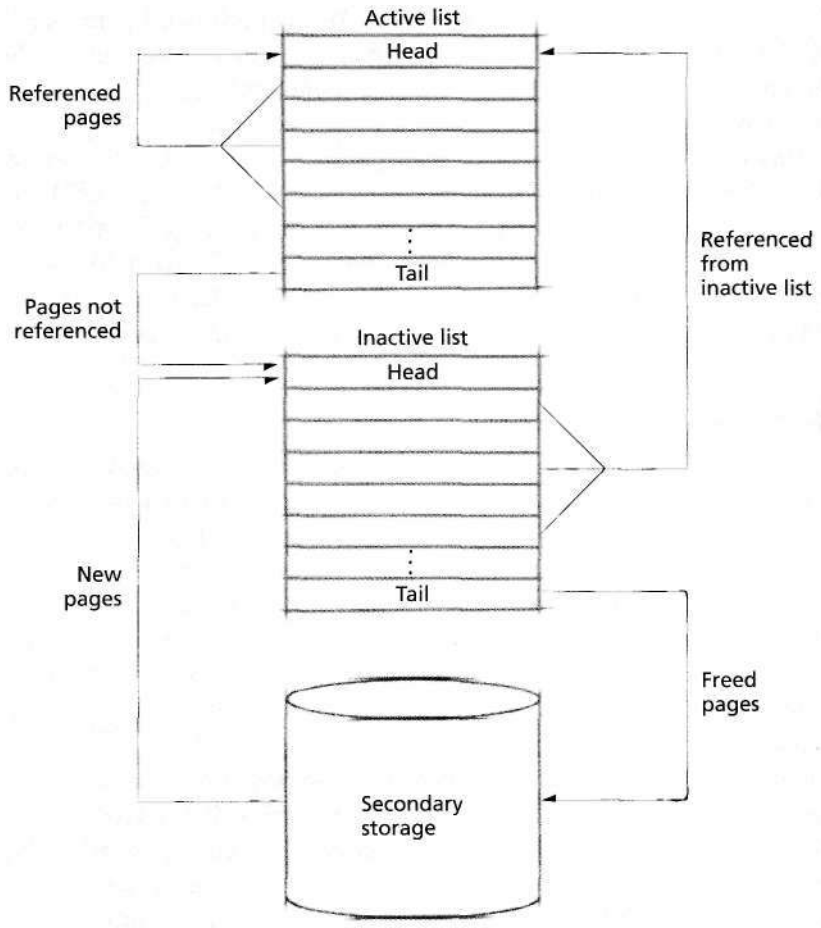

Figure 20.9 | *Page-replacement system overview.*